

Big Data

Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered. Big data often comes from multiple sources and arrives in multiple formats.

Some key points to consider when studying Big Data are:

1. Big data can be analyzed for insights that lead to better decisions and strategic business moves.
2. The primary goal of big data analytics is to help companies make more informed business decisions by enabling data scientists, predictive modelers, and other analytics professionals to analyze large volumes of transaction data, as well as other forms of data that may be untapped by conventional business intelligence (BI) programs.
3. Big data technologies, like Hadoop and cloud-based analytics, bring significant cost advantages when it comes to storing large amounts of data.
4. Big data analytics can help organizations to harness their data and use it to identify new opportunities, leading to smarter business moves, more efficient operations, higher profits, and happier customers.
5. Big data is characterized by the 3Vs: the extreme volume of data, the wide variety of data types, and the velocity at which the data must be processed.

Unit 1 - Introduction to Big Data

1. **Definition:** Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered.
2. **Characteristics:** Big data is often characterized by the 3Vs: Volume, Velocity, and Variety. Volume refers to the large amount of data, Velocity refers to the speed at which data is generated and processed, and Variety refers to the different types of data.
3. **Sources:** Big data can come from various sources such as social media, business transactions, machine-generated data, and government records.
4. **Challenges:** The main challenges of big data include storage, analysis, and visualization. Storing and managing large amounts of data can be difficult and expensive. Analyzing big data requires advanced tools and techniques, and visualizing the results in a meaningful way can be challenging.
5. **Benefits:** Big data can provide valuable insights and help organizations make better decisions. It can be used to improve customer experience,

optimize operations, and reduce costs.

6. **Applications:** Big data has numerous applications in various industries such as healthcare, finance, and retail. It can be used for predictive analytics, fraud detection, and personalized marketing.
7. **Future:** The field of big data is constantly evolving, with new technologies and techniques being developed to handle the growing volume, velocity, and variety of data. The future of big data is likely to involve more advanced analytics, machine learning, and artificial intelligence.

Types of digital data in big data

Big data refers to the large and complex sets of digital data that are generated from various sources and in various formats. These data sets are so large and complex that traditional data processing methods are inadequate to handle them. The following are the types of digital data in big data:

1. **Structured data:** This type of data is organized in a predefined manner and can be easily processed and analyzed. Examples of structured data include data stored in relational databases, spreadsheets, and CSV files.
2. **Unstructured data:** This type of data has no predefined structure and is difficult to process and analyze. Examples of unstructured data include text, images, videos, and audio files.
3. **Semi-structured data:** This type of data has some structure but is not as organized as structured data. Examples of semi-structured data include XML and JSON files.
4. **Time-series data:** This type of data is collected over time and is used to track changes and trends. Examples of time-series data include stock prices, weather data, and sensor data.
5. **Geospatial data:** This type of data is used to represent geographical locations and features. Examples of geospatial data include maps, satellite images, and GPS data.
6. **Multimedia data:** This type of data includes images, videos, and audio files. Multimedia data is often unstructured and can be difficult to process and analyze.
7. **Web and social media data:** This type of data is generated from web and social media platforms and includes text, images, videos, and user-generated content.

These are some of the common types of digital data in big data. Each type of data has its own unique characteristics and requires specific methods for processing and analysis. Understanding the different types of data is essential for effectively managing and analyzing big data.

History of Big Data Innovation

- The history of Big Data as a term may be brief, but many of the foundations it is built on were laid long ago .
- Before 1980, companies used mainframe computers to store and analyze data. Two key technologies were critical to the first formations of data centers as we think of them today .
- In 1965, the US Government planned the world’s first data center to store 742 million tax returns and 175 million sets of fingerprints on magnetic tape .
- In 1970, IBM mathematician Edgar F Codd presented his framework for a “relational database” .
- Data became a problem for the U.S. Census Bureau in 1880. They estimated it would take eight years to handle and process the data collected during the 1880 census, and predicted the data from the 1890 census would take more than 10 years to process .
- Big data has revolutionized the modern business environment in recent years. A mixture of structured, semistructured, and unstructured data, big data is a collection of information that organizations can mine for business purposes through machine learning, predictive modeling, and other advanced data analytics applications .

Introduction to Big Data Platform

1. Big Data refers to the large and complex sets of data that are difficult to process using traditional data processing applications.
2. Big Data platforms are designed to handle, store, and analyze these large datasets in a more efficient and scalable manner.
3. These platforms often use distributed computing and storage systems, such as Hadoop and Spark, to process and analyze the data.
4. Big Data platforms can handle structured, semi-structured, and unstructured data, and can be used for a variety of applications, including data mining, machine learning, and predictive analytics.
5. The use of Big Data platforms has become increasingly popular in recent years, as organizations seek to gain insights from their data and make data-driven decisions.
6. There are many different Big Data platforms available, each with its own strengths and capabilities. It is important to carefully evaluate and choose the right platform for your specific needs and requirements.

Drivers for Big Data

1. **Volume:** The amount of data being generated and stored is growing at an exponential rate, driven by the increasing use of digital devices and the Internet of Things (IoT).
2. **Velocity:** The speed at which data is being generated and processed is also increasing, driven by the need for real-time analysis and decision-

making.

3. **Variety:** Data is being generated in a wide variety of formats, including structured, semi-structured, and unstructured data, driven by the increasing use of social media, mobile devices, and other digital platforms.
4. **Veracity:** The accuracy and reliability of data is becoming increasingly important, driven by the need for data-driven decision-making and the potential consequences of inaccurate or unreliable data.
5. **Value:** The potential value that can be derived from big data is driving its adoption, as organizations seek to gain insights and make data-driven decisions to improve their operations and competitiveness.

Big Data Architecture

Big Data Architecture refers to the design and implementation of a system that can handle large volumes of data. It involves the use of various technologies and tools to store, process, and analyze data. Some key points to consider when designing a Big Data Architecture are:

1. **Data Ingestion:** This refers to the process of collecting and importing data into the system. Data can come from various sources such as logs, social media, sensors, etc. and can be in different formats such as structured, semi-structured, or unstructured.
2. **Data Storage:** This refers to the storage of data in a way that it can be easily accessed and processed. Data can be stored in various formats such as relational databases, NoSQL databases, data lakes, etc.
3. **Data Processing:** This refers to the processing of data to extract insights and knowledge. Data processing can be done using various tools and technologies such as MapReduce, Spark, etc.
4. **Data Analysis:** This refers to the analysis of data to extract insights and knowledge. Data analysis can be done using various tools and technologies such as machine learning, data mining, etc.
5. **Data Visualization:** This refers to the visualization of data to make it easier to understand and interpret. Data visualization can be done using various tools and technologies such as charts, graphs, etc.
6. **Scalability:** Big Data Architecture should be designed in a way that it can handle the increasing volume of data. This can be achieved by using distributed systems, cloud computing, etc.
7. **Security:** Big Data Architecture should be designed in a way that it ensures the security and privacy of data. This can be achieved by using encryption, access control, etc.

In summary, Big Data Architecture involves the use of various technologies and tools to handle large volumes of data. It is important to consider factors such as

data ingestion, data storage, data processing, data analysis, data visualization, scalability, and security when designing a Big Data Architecture.

Big data characteristics

Big data is a term used to describe large, complex datasets that traditional data processing methods are unable to handle. The characteristics of big data are commonly referred to as the “5 Vs”:

1. **Volume:** The sheer amount of data being generated and stored. This can range from terabytes to petabytes and beyond.
2. **Velocity:** The speed at which data is being generated and processed. This can include real-time data streams and the need for fast processing and analysis.
3. **Variety:** The different types of data being generated, including structured, unstructured, and semi-structured data.
4. **Veracity:** The quality and accuracy of the data. This can include issues with data inconsistency, incompleteness, and ambiguity.
5. **Value:** The potential value that can be derived from the data through analysis and processing.

These characteristics present both challenges and opportunities for organizations looking to harness the power of big data. By developing the right tools and techniques to manage and analyze big data, organizations can gain valuable insights and make more informed decisions.

5 Vs of Big Data

The 5 Vs of Big Data refer to the five key characteristics that define Big Data: Volume, Velocity, Variety, Veracity, and Value.

1. **Volume:** This refers to the vast amount of data that is generated every second. With the rise of the Internet of Things (IoT) and social media, the volume of data being generated is growing at an exponential rate.
2. **Velocity:** This refers to the speed at which data is generated and processed. With the increasing use of real-time data, the velocity of data processing has become a critical factor in many industries.
3. **Variety:** This refers to the different types of data that are being generated. Data can come in many forms, including structured, semi-structured, and unstructured data.
4. **Veracity:** This refers to the accuracy and reliability of the data. With the increasing amount of data being generated, it is important to ensure that the data is accurate and can be trusted.
5. **Value:** This refers to the ability to extract valuable insights from the data. With the increasing amount of data being generated, it is important to

have the ability to analyze the data and extract valuable insights that can be used to make informed decisions.

These 5 Vs of Big Data are important characteristics to consider when working with Big Data. Understanding these characteristics can help organizations to effectively manage and analyze their data.

Big Data technology components

Big Data technology typically refers to three viewpoints of technical innovation and super-large datasets: automated parallel computation, data management schemes, and data mining. A typical Big Data technology stack consists of numerous layers, namely data analytics, data modeling, data warehousing, and the data pipeline layer. Each of these is interdependent and plays a crucial and unique role, ensuring the smooth functioning of the entire stack of technologies.

Some of the main components of Big Data technology include:

1. **Machine Learning**
2. **Natural Language Processing (NLP)**
3. **Business Intelligence**
4. **Cloud Computing**
5. **Data Sources:** All big data solutions start with one or more data sources. Examples include application data stores, such as relational databases, and static files produced by applications.

It is important to note that the specific components and their arrangement may vary depending on the specific needs and goals of the organization implementing the Big Data technology stack.

Big Data importance

Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered. Big data often comes from multiple sources and arrives in multiple formats.

The importance of big data lies in its ability to provide insights and improve decision-making in various fields. Some of the key benefits of big data include:

1. **Improved efficiency and productivity:** Big data can help organizations streamline their operations and increase efficiency by identifying areas for improvement and optimization.
2. **Better decision-making:** Big data can provide valuable insights that can inform and improve decision-making in areas such as marketing, sales, and product development.

3. **Enhanced customer experience:** Big data can help organizations better understand their customers and provide a more personalized and satisfying experience.
4. **Innovation:** Big data can drive innovation by providing new insights and revealing previously unknown patterns and relationships.
5. **Competitive advantage:** Organizations that can effectively harness the power of big data can gain a competitive advantage over their rivals by making better decisions, improving efficiency, and enhancing the customer experience.

Overall, the importance of big data lies in its ability to provide valuable insights and improve decision-making in a wide range of fields. By effectively harnessing the power of big data, organizations can gain a competitive edge and drive innovation and growth.

Big Data applications

Big Data refers to the large and complex data sets that are generated from various sources and are difficult to process using traditional data processing tools. Big Data applications are used to process, analyze, and extract insights from these data sets. Some of the common applications of Big Data are:

1. **Business Intelligence and Analytics:** Big Data is used to analyze customer behavior, market trends, and sales patterns to make informed business decisions.
2. **Fraud Detection:** Big Data is used to identify fraudulent activities in various industries such as banking, insurance, and e-commerce.
3. **Healthcare:** Big Data is used to analyze patient data to improve healthcare outcomes, reduce costs, and enhance patient experiences.
4. **Smart Cities:** Big Data is used to improve urban planning, transportation, and public safety in smart cities.
5. **Internet of Things (IoT):** Big Data is used to analyze data generated from IoT devices to improve operational efficiency and enhance customer experiences.
6. **Social Media:** Big Data is used to analyze social media data to understand customer behavior, preferences, and sentiments.
7. **Supply Chain Management:** Big Data is used to optimize supply chain operations, reduce costs, and improve customer satisfaction.

These are just a few examples of how Big Data is being used across various industries. The potential applications of Big Data are vast and are constantly evolving.

Big Data features – security, compliance, auditing and protection

Big data refers to the large and complex data sets that are generated by businesses, governments, and other organizations. These data sets are often too large to be processed using traditional data processing methods, and require specialized tools and techniques to manage and analyze. As the use of big data continues to grow, it is important to consider the security, compliance, auditing, and protection features that are necessary to ensure the integrity and confidentiality of the data.

1. **Security:** Big data security involves protecting the data from unauthorized access, use, disclosure, disruption, modification, or destruction. This can be achieved through a variety of measures, including encryption, access controls, and network security.
2. **Compliance:** Compliance refers to the need to adhere to legal and regulatory requirements related to the collection, storage, and use of big data. This includes complying with data protection and privacy laws, as well as industry-specific regulations.
3. **Auditing:** Auditing involves regularly reviewing and monitoring the use of big data to ensure that it is being used in accordance with established policies and procedures. This can help to identify any potential security risks or compliance issues, and can also provide valuable insights into how the data is being used.
4. **Protection:** Protection involves implementing measures to safeguard the integrity and availability of big data. This can include measures such as data backup and recovery, as well as disaster recovery planning to ensure that the data can be restored in the event of a system failure or other disruption.

In summary, big data presents a number of challenges when it comes to security, compliance, auditing, and protection. By implementing appropriate measures and regularly reviewing and monitoring the use of big data, organizations can help to ensure that their data is secure, compliant, and protected.

Security of Big Data

1. **Encryption:** One of the most important methods to secure big data is through encryption. This involves encoding the data in such a way that only authorized parties can access it.
2. **Access Control:** Access control is another important aspect of big data security. This involves setting up permissions and restrictions on who can access the data and what they can do with it.
3. **Data Masking:** Data masking is a technique used to protect sensitive data by replacing it with fake data that looks and behaves like the original data.

4. **Backup and Recovery:** It is important to have a backup and recovery plan in place to ensure that data can be recovered in the event of a disaster or data loss.
5. **Auditing and Monitoring:** Regular auditing and monitoring of big data systems can help detect and prevent security breaches.
6. **Regular Updates and Patches:** It is important to keep big data systems up to date with the latest security patches and updates to prevent vulnerabilities from being exploited.
7. **Employee Training:** Employee training is an important aspect of big data security. Employees should be trained on how to handle sensitive data and how to identify and prevent security threats.
8. **Physical Security:** Physical security measures such as locked doors and surveillance cameras can help prevent unauthorized access to big data systems.

These are some of the key points to consider when it comes to the security of big data. It is important to have a comprehensive security plan in place to protect against potential threats.

Compliance of Big Data

1. **Data Protection and Privacy:** Organizations must ensure that they comply with data protection and privacy regulations when handling big data. This includes obtaining consent from individuals before collecting and using their personal data, and ensuring that the data is stored securely and used only for the purposes for which it was collected.
2. **Data Retention:** Organizations must comply with data retention regulations, which specify how long data must be kept and when it must be deleted. This is particularly important for sensitive data, such as financial or medical records.
3. **Data Quality:** Organizations must ensure that the data they collect and use is accurate, complete, and up-to-date. This is important for making informed decisions and avoiding errors.
4. **Data Governance:** Organizations must have a data governance framework in place to ensure that data is managed in a responsible and ethical manner. This includes having clear policies and procedures for data collection, storage, use, and sharing.
5. **Transparency:** Organizations must be transparent about their data collection and use practices. This includes providing clear and concise information to individuals about how their data is being used, and giving them the ability to access, correct, or delete their personal data.

6. **Accountability:** Organizations must be accountable for their data practices and must be able to demonstrate compliance with data protection and privacy regulations. This includes having regular audits and assessments to ensure that data is being handled in a responsible and compliant manner.
7. **Ethical Use of Data:** Organizations must ensure that they use data in an ethical manner, avoiding practices that could harm individuals or society. This includes avoiding discriminatory or biased decision-making based on data, and ensuring that data is used for the benefit of society as a whole.

Auditing of Big Data

1. **Introduction:** Big data refers to the large and complex data sets that are generated by businesses, governments, and other organizations. These data sets are often too large to be processed using traditional data processing techniques, and require specialized tools and techniques to manage and analyze.
2. **Challenges in auditing big data:** Auditing big data presents several challenges, including the volume, velocity, and variety of the data. Auditors must be able to handle large amounts of data, process it quickly, and deal with a wide range of data types and formats.
3. **Techniques for auditing big data:** There are several techniques that can be used to audit big data, including data sampling, data profiling, and data analytics. These techniques can help auditors to identify anomalies, trends, and patterns in the data, and to assess the accuracy and completeness of the data.
4. **Tools for auditing big data:** There are several tools available to help auditors to manage and analyze big data, including data visualization tools, data mining tools, and data analysis tools. These tools can help auditors to extract insights from the data, and to present their findings in a clear and concise manner.
5. **Conclusion:** Auditing big data is a complex and challenging task, but it is essential for ensuring the accuracy and completeness of the data. By using the right techniques and tools, auditors can effectively manage and analyze big data, and provide valuable insights to their clients.

Protection of Big Data

Big data includes big privacy concerns. To safeguard big data and ensure it can be used for analytics, you need to create a framework for privacy protection that can handle the volume, velocity, variety, and value of big data as it is moved between environments, processed, analyzed, and shared.

The key to protecting the privacy of your big data while still optimizing its value is ongoing review of four critical data management activities : 1. **Data collection** 2. **Retention and archiving** 3. **Data use**, including use in testing, DevOps, and other data masking scenarios 4. **Creating and updating disclosure policies and practices**

Big data security's mission is clear enough: keep out on unauthorized users and intrusions with firewalls, strong user authentication, end-user training, and intrusion protection systems (IPS) and intrusion detection systems (IDS). In case someone does gain access, encrypt your data in transit and at rest.

Big Data privacy

1. **Definition:** Big Data privacy refers to the protection of personal information collected, processed, and stored in large datasets.
2. **Challenges:** The sheer volume and complexity of Big Data can make it difficult to ensure the privacy of individuals. Additionally, the use of advanced analytics and machine learning algorithms can reveal sensitive information about individuals, even if the data is anonymized.
3. **Regulations:** Governments around the world have introduced regulations to protect the privacy of individuals in the context of Big Data. For example, the European Union's General Data Protection Regulation (GDPR) sets strict standards for the collection, use, and storage of personal data.
4. **Best practices:** Organizations can adopt best practices to protect the privacy of individuals in the context of Big Data. These include data minimization, transparency, and user control over their personal data.
5. **Future developments:** As the use of Big Data continues to grow, it is likely that new challenges and opportunities will arise in the area of privacy. It is important for organizations to stay up-to-date with the latest developments and to continue to prioritize the protection of personal data.

Big Data Ethics

Big data ethics, also known as data ethics, refers to the systematization, defense, and recommendation of concepts of right and wrong conduct in relation to data, particularly personal data . Since the dawn of the Internet, the quantity and quality of data have dramatically increased and continue to do so exponentially .

Big data ethics differs from information ethics in that the focus of information ethics is more concerned with issues of intellectual property and concerns relating to librarians, archivists, and information professionals. In contrast, big data ethics is more concerned with collectors and disseminators of structured or unstructured data, such as data brokers, governments, and large corporations .

Data ethics is concerned with the following principles : - **Ownership**: Individuals own their own data. - **Transaction transparency**: If an individual's personal data is used, they should have transparent access to the transaction. - **Consent**: If an individual or legal entity would like to use personal data, one must obtain consent.

A strong data ethics program requires a specific team or role to focus on ethics issues; it cannot be a side activity .

Big Data Analytics

Big Data Analytics refers to the process of collecting, organizing, and analyzing large sets of data, known as Big Data, to discover patterns and other useful information. Big Data Analytics can help organizations to better understand the information contained within the data and will also help identify the data that is most important to the business and future business decisions.

Some key points to consider when studying Big Data Analytics are:

1. Big Data is a term used to describe the large volume of structured and unstructured data that is generated by businesses, social media, and other sources.
2. Big Data Analytics is used to extract meaningful insights from the data, which can help organizations make informed decisions.
3. Big Data Analytics involves the use of advanced analytical techniques, such as machine learning, data mining, and predictive analytics.
4. Big Data Analytics can be applied to various industries, including health-care, finance, and retail, to improve business operations and customer experiences.
5. Big Data Analytics requires the use of specialized tools and technologies, such as Hadoop and Spark, to process and analyze the data.
6. Big Data Analytics can help organizations to gain a competitive advantage by providing insights into customer behavior, market trends, and operational efficiency.
7. Big Data Analytics is a rapidly growing field, with a high demand for skilled professionals who can work with large datasets and advanced analytical tools.
8. Big Data Analytics can present challenges, such as data privacy and security, and requires careful management to ensure that the data is used ethically and responsibly.

In summary, Big Data Analytics is a powerful tool that can help organizations to gain valuable insights from large datasets, and can be applied to a wide range of industries to improve business operations and customer experiences. It

is important to have a strong understanding of the tools and techniques used in Big Data Analytics, as well as the ethical considerations involved in working with large datasets.

Challenges of conventional systems compared to Big Data

1. **Scalability:** Conventional systems are not designed to handle the large volume of data that Big Data systems can handle. As the volume of data grows, conventional systems struggle to scale and process the data efficiently.
2. **Data Variety:** Big Data systems are designed to handle a wide variety of data types, including structured, semi-structured, and unstructured data. Conventional systems, on the other hand, are typically designed to handle structured data and may struggle to process and analyze data that does not fit into a predefined schema.
3. **Real-time Processing:** Big Data systems are designed to process data in real-time, allowing for faster decision making and more timely insights. Conventional systems, on the other hand, may require batch processing, which can result in delays and outdated information.
4. **Cost:** Big Data systems can be more cost-effective than conventional systems, as they are designed to run on commodity hardware and can scale horizontally by adding more machines to the cluster. Conventional systems, on the other hand, may require expensive, specialized hardware and may not scale as easily.
5. **Flexibility:** Big Data systems are designed to be flexible and can adapt to changing data and processing requirements. Conventional systems, on the other hand, may be more rigid and may require significant effort to modify or update.

These are some of the challenges that conventional systems face when compared to Big Data systems. Big Data systems are designed to handle large volumes of data, process data in real-time, and adapt to changing requirements, making them a more suitable choice for many modern data processing needs.

Intelligent Data Analysis in Big Data

Intelligent data analysis is a process of analyzing and interpreting large and complex data sets, known as Big Data, using advanced computational and statistical techniques. The goal of intelligent data analysis is to extract meaningful insights and knowledge from the data, which can be used to inform decision-making and drive business value.

Some key points to consider when performing intelligent data analysis in Big Data include:

1. **Data Preprocessing:** Before analysis can begin, the data must be pre-processed to ensure its quality and usability. This may involve cleaning, transforming, and normalizing the data to remove any inconsistencies or errors.
2. **Data Mining:** Data mining involves the use of algorithms and statistical models to identify patterns and relationships within the data. This can help to uncover hidden insights and knowledge that would otherwise be difficult to detect.
3. **Machine Learning:** Machine learning is a subset of artificial intelligence that involves the use of algorithms to automatically learn from data and make predictions or decisions. Machine learning can be used to build predictive models, classify data, and identify anomalies within the data.
4. **Visualization:** Visualization is an important tool for exploring and understanding data. By creating visual representations of the data, it is possible to identify trends, patterns, and relationships that may not be immediately apparent from the raw data.
5. **Scalability:** Big Data can be extremely large and complex, and traditional data analysis techniques may not be able to handle the scale of the data. It is important to use scalable techniques and technologies that can efficiently process and analyze large volumes of data.

Intelligent data analysis in Big Data is a powerful tool for extracting valuable insights and knowledge from large and complex data sets. By using advanced computational and statistical techniques, it is possible to uncover hidden patterns and relationships within the data, and use this information to drive business value.

Nature of Data in Big Data

1. **Volume:** Big Data is characterized by the large volume of data that is generated and stored. This volume of data is typically measured in terabytes or petabytes.
2. **Variety:** Big Data includes data from a wide variety of sources, including structured data from traditional databases, unstructured data such as text, images, and videos, and semi-structured data such as XML and JSON.
3. **Velocity:** Big Data is generated at a high velocity, with data being created and processed in real-time or near real-time.
4. **Veracity:** Big Data can be uncertain or incomplete, and the quality of the data can vary. This can be due to errors in data collection, data entry, or data processing.
5. **Value:** Big Data has the potential to provide valuable insights and drive decision-making, but extracting value from Big Data requires the use of

advanced analytics techniques.

6. **Complexity:** Big Data can be complex, with data coming from multiple sources and in multiple formats. This can make it challenging to process and analyze the data.

Analytic Processes and Tools for Big Data

Big data analytics is the process of examining large and varied data sets to uncover hidden patterns, unknown correlations, market trends, customer preferences, and other useful information. Here are some of the analytic processes and tools used for big data:

1. **Data Mining:** Data mining is the process of discovering patterns and knowledge from large amounts of data. The data sources can include databases, data warehouses, the internet, and other sources.
2. **Machine Learning:** Machine learning is a type of artificial intelligence that allows software applications to become more accurate in predicting outcomes without being explicitly programmed to do so. Machine learning algorithms use historical data as input to predict new output values.
3. **Predictive Analytics:** Predictive analytics is the practice of extracting information from existing data sets to determine patterns and predict future outcomes and trends. Predictive analytics techniques include data mining, machine learning, and statistical modeling.
4. **Hadoop:** Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware. It provides massive storage for any kind of data, enormous processing power, and the ability to handle virtually limitless concurrent tasks or jobs.
5. **NoSQL Databases:** NoSQL databases are non-relational databases that are used to store and retrieve data. They are often used for big data and real-time web applications. NoSQL databases can handle large volumes of structured, semi-structured, or unstructured data.
6. **Data Visualization:** Data visualization is the presentation of data in a graphical or pictorial format. It enables decision-makers to see analytics presented visually, so they can grasp difficult concepts or identify new patterns.

These are some of the analytic processes and tools used for big data. They can help organizations to analyze large and complex data sets, and make informed decisions based on the insights gained.

Analysis vs Reporting in Big Data

- **Analysis** in Big Data refers to the process of examining large and complex data sets to uncover hidden patterns, correlations, and other insights. It

involves using advanced analytical techniques and tools to make sense of the data and extract valuable information.

- **Reporting** in Big Data, on the other hand, refers to the process of organizing and presenting data in a way that is easy to understand and interpret. It involves creating visualizations, dashboards, and reports that summarize the data and highlight key findings.
- The main difference between analysis and reporting in Big Data is the level of complexity involved. Analysis is a more complex process that requires specialized skills and knowledge, while reporting is more straightforward and can be done by anyone with basic data visualization skills.
- Another key difference is the purpose of each process. Analysis is done to gain a deeper understanding of the data and uncover hidden insights, while reporting is done to communicate the findings to others.
- Both analysis and reporting are important components of Big Data, and they complement each other. Analysis helps to uncover valuable insights, while reporting helps to communicate those insights to others.
- In summary, analysis and reporting in Big Data are two distinct but complementary processes. Analysis involves examining complex data sets to uncover hidden insights, while reporting involves organizing and presenting the data in a way that is easy to understand. Both processes are important for making the most of Big Data.

Modern Data Analytic Tools for Big Data

Big data analytics is the process of examining large and varied data sets to uncover hidden patterns, unknown correlations, market trends, customer preferences, and other useful information. Here are some modern data analytic tools for Big Data:

1. **R-Programming:** R programming is one of the best big data analytics tools free for everyone.
2. **Apache Hadoop:** Apache Hadoop is the leading big data analytics tool open source. It is a Java-based free software framework that helps in the effective storage of a huge amount of data in a storage place known as a cluster.
3. **MongoDB:** MongoDB is the leading database software in the world. It is used to analyze data fast and efficiently.
4. **RapidMiner:** RapidMiner is one of the leading platforms for analysts to integrate data prep, machine learning, predictive model deployment, etc.
5. **Apache Spark:** Apache Spark is another popular big data analytics tool.
6. **Microsoft Power BI:** Microsoft Power BI is best for data visualization.
7. **Tableau:** Tableau is best for business intelligence.
8. **Qlik Sense:** Qlik Sense is best for machine learning.
9. **Looker:** Looker is best for data exploration.

10. **Klipfolio:** Klipfolio is best for instant metrics.

These are some of the most popular and reliable modern data analytic tools for Big Data. They can help you uncover hidden patterns, unknown correlations, market trends, customer preferences, and other useful information from large and varied data sets.

Unit 2 - Hadoop and Map Reduce

- **Hadoop** is an open-source software framework for storing and processing large datasets using distributed computing.
- It is designed to scale up from a single server to thousands of machines, each offering local computation and storage.
- Hadoop consists of two main components: the **Hadoop Distributed File System (HDFS)** and the **MapReduce** programming model.
- HDFS is a distributed file system that provides high-throughput access to application data. It is designed to be fault-tolerant and to run on commodity hardware.
- **MapReduce** is a programming model for processing large datasets in parallel across a cluster of machines.
- In the MapReduce model, the data is divided into chunks and processed by the **map** function, which produces a set of intermediate key-value pairs.
- These intermediate key-value pairs are then shuffled and sorted by the framework and fed into the **reduce** function, which produces the final output.
- Hadoop and MapReduce are widely used in big data processing, including data mining, machine learning, and data analysis.

Hadoop

- Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware .
- It provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs .
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- In this way, Hadoop can efficiently store and process large datasets ranging in size from gigabytes to petabytes of data .
- It is used to manage data, store data, and process data for various big data applications running under clustered systems .
- The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models .
- It is designed to scale up from single servers to thousands of machines, each offering local computation and storage .

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation .
- It provides a software framework for distributed storage and processing of big data using the MapReduce programming model .

History of Hadoop

- Hadoop was started with Doug Cutting and Mike Cafarella in the year 2002 when they both started to work on Apache Nutch project. Apache Nutch project was the process of building a search engine system that can index 1 billion pages .
- Hadoop was named after an extinct specie of mammoth, a so called Yellow Hadoop .
- Hadoop is an open-source software framework for storing and processing big data. It was created by Apache Software Foundation in 2006, based on a white paper written by Google in 2003 that described the Google File System (GFS) and the MapReduce programming model .
- In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data .
- Hadoop YARN was introduced in 2012, a platform responsible for managing computing resources in clusters and using them for scheduling users' applications .
- Hadoop Ozone was introduced in 2020, an object store for Hadoop .

Apache Hadoop

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation.
- It provides a software framework for distributed storage and processing of big data using the MapReduce programming model.
- The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- It is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- The ApacheTM Hadoop® project develops open-source software for reliable, scalable, distributed computing.

Hadoop Distributed File System

- The Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware.
- It is highly fault-tolerant and is designed to be deployed on low-cost hardware.

- HDFS provides high throughput access to application data and is suitable for applications that have large data sets.
- HDFS stores large files across multiple machines and achieves reliability by replicating the data across multiple hosts.
- HDFS is designed to work with MapReduce, a software framework for distributed processing of large data sets.
- HDFS has a master/slave architecture. An HDFS cluster consists of a single NameNode, a master server that manages the file system namespace and regulates access to files by clients.
- In addition, there are a number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on.
- HDFS exposes a file system namespace and allows user data to be stored in files.
- Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.
- The NameNode executes file system namespace operations like opening, closing, and renaming files and directories.
- It also determines the mapping of blocks to DataNodes.
- The DataNodes are responsible for serving read and write requests from the file system's clients.
- The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

Components of Hadoop

Hadoop is an open-source software framework for storing and processing large datasets. It is designed to scale up from a single server to thousands of machines, each providing local computation and storage. The core components of Hadoop are:

1. **Hadoop Distributed File System (HDFS):** A distributed file system that provides high-throughput access to application data. It is designed to store and manage large files across multiple machines.
2. **MapReduce:** A programming model for processing large datasets in parallel across a Hadoop cluster. It divides the input data into smaller chunks and assigns them to different nodes for processing.
3. **YARN (Yet Another Resource Negotiator):** A resource management layer that schedules and allocates resources to applications running on the Hadoop cluster.
4. **Hadoop Common:** A set of utilities and libraries that support the other Hadoop components. It provides the basic infrastructure and tools required for running Hadoop.

These components work together to provide a reliable and scalable platform for storing and processing large datasets. Hadoop is widely used in big data

applications, including data mining, machine learning, and predictive analytics.

Data Format Co.

- Data format refers to the way in which data is organized and stored.
- It is important to choose the right data format for a specific application, as it can affect the efficiency of data processing and retrieval.
- Common data formats include CSV, JSON, XML, and binary formats.
- CSV (Comma-Separated Values) is a simple text-based format that stores data in rows and columns, separated by commas.
- JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.
- XML (eXtensible Markup Language) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.
- Binary formats store data in a compact, binary representation, which can be more efficient for storage and processing, but may not be as human-readable as text-based formats.
- Choosing the right data format depends on factors such as the type of data being stored, the intended use of the data, and the requirements for data processing and retrieval.

Analyzing Data with Hadoop

Hadoop is an open-source software framework that allows for the distributed processing of large datasets across clusters of computers using simple programming models. Here are some key points to consider when analyzing data with Hadoop:

1. Hadoop is designed to scale up from a single server to thousands of machines, each offering local computation and storage.
2. Hadoop uses a distributed file system called Hadoop Distributed File System (HDFS) to store data across multiple machines.
3. Hadoop uses MapReduce, a programming model for processing large datasets, to analyze data.
4. Hadoop can be used with a variety of data sources, including structured, semi-structured, and unstructured data.
5. Hadoop can be used for a wide range of data analysis tasks, including data mining, machine learning, and predictive analytics.
6. Hadoop is highly flexible and can be customized to meet the specific needs of an organization.
7. Hadoop is widely used in industries such as finance, healthcare, retail, and telecommunications to analyze large datasets.

Scaling out with Hadoop

- Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers.
- It is designed to scale up from a single server to thousands of machines, each offering local computation and storage.
- Hadoop achieves reliability by replicating data across multiple nodes.
- The Hadoop Distributed File System (HDFS) is the primary storage system used by Hadoop applications.
- Hadoop uses a programming model called MapReduce to process and generate large data sets.
- Hadoop can be used to scale out data processing tasks by distributing them across a cluster of computers.
- This allows for faster processing times and the ability to handle larger data sets.
- Hadoop can be used in a variety of applications, including data mining, machine learning, and predictive analytics.
- Hadoop is widely used by companies such as Facebook, Yahoo, and Amazon to process large amounts of data.
- Hadoop is a powerful tool for scaling out data processing tasks and can help organizations to handle large amounts of data more efficiently.

Hadoop Streaming

- Hadoop Streaming is a utility that comes with the Hadoop distribution.
- It allows you to create and run Map/Reduce jobs with any executable or script as the mapper and/or the reducer.
- Hadoop Streaming is particularly useful for processing unstructured data, such as text, where the data must be parsed in an ad-hoc manner.
- The utility allows you to use languages other than Java for MapReduce programming.
- Hadoop Streaming works by redirecting standard input and output of the mapper and reducer programs to the Hadoop framework.
- The input data is passed to the mapper program via standard input, and the output of the mapper is passed to the reducer via standard output.
- The utility takes care of all the details of data partitioning, sorting, and shuffling between the mapper and the reducer.
- To use Hadoop Streaming, you need to specify the mapper and reducer programs using the **-mapper** and **-reducer** command line options, respectively.
- You can also specify additional options, such as the input and output directories, the input and output formats, and the number of reduce tasks.
- Hadoop Streaming is a powerful and flexible tool for processing large datasets using a variety of programming languages and tools.

Hadoop Pipes

- Hadoop Pipes is the name of the C++ interface to Hadoop MapReduce.

- Unlike Streaming, which uses standard input and output to communicate with the map and reduce code, Pipes uses sockets as the channel over which the tasktracker communicates with the process running the C++ map or reduce function.
- JNI is not used.
- Hadoop Pipes uses sockets to enable tasktrackers to communicate processes running the C++ map or reduce functions.
- To run Hadoop pipes on IBM PowerLinux systems, build your Hadoop version for PowerLinux and compile the pipes libraries as described in step 1.
- Hadoop MapReduce is a software framework for easily writing applications which process vast amounts of data (multi-terabyte data-sets) in-parallel on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner.
- If it weren't for the size of the records, the Hadoop Pipes API would be fine: but this seems to be based around passing the input to map/reduce tasks as a contiguous block of bytes, which is impractical in this case.

Hadoop Ecosystem

The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop. Hadoop is an ecosystem of open source components that fundamentally changes the way enterprises store, process, and analyze data. Unlike traditional systems, Hadoop enables multiple types of analytic workloads to run on the same data, at the same time, at massive scale on industry-standard hardware.

The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage.

Some of the components that collectively form a Hadoop ecosystem are: - **HDFS**: Hadoop Distributed File System - **YARN**: Yet Another Resource Negotiator - **MapReduce**: Programming based Data Processing - **Spark**: In-Memory data processing - **PIG, HIVE**: Query based processing of data services - **HBase**: NoSQL Database

The Hadoop ecosystem is one of the most critical developments in Big Data. It provides tools for storing and analyzing data and a framework for other companies to develop their applications.

Map Reduce

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It is designed to allow for distributed processing of large data sets across a cluster of computers. The key concepts of MapReduce are:

1. **Map:** This step involves taking a set of data and converting it into another set of data, where individual elements are broken down into tuples (key/value pairs).
2. **Reduce:** This step involves taking the output from the Map step and combining those data tuples into a smaller set of tuples.
3. **Input and Output:** The input and output of the MapReduce job are both stored in a distributed file system, such as Hadoop Distributed File System (HDFS).
4. **Partitioning:** The MapReduce framework partitions the input data into a set of M splits, and the map function is applied to each split in parallel.
5. **Shuffling:** The MapReduce framework redistributes the data among the reducers based on the output keys produced by the map function.
6. **Combiner:** The combiner function is an optional step that can be used to perform local aggregation of the intermediate data generated by the map function.

MapReduce is widely used for big data processing, such as data mining, data analysis, and machine learning. It is a powerful tool for handling large-scale data processing tasks.

Map Reduce framework and basics

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It is designed to handle large volumes of data in a parallel and distributed manner.

1. The MapReduce framework consists of two main functions: Map and Reduce.
2. The Map function takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs).
3. The Reduce function takes the output from the Map function and combines those data tuples into a smaller set of tuples.
4. The MapReduce framework is designed to be fault-tolerant and scalable, allowing for the processing of large amounts of data across multiple machines.
5. The framework is commonly used for data-intensive tasks such as search, data mining, and log analysis.
6. MapReduce is often used in conjunction with the Hadoop Distributed File System (HDFS) for distributed data storage and processing.
7. The framework is widely used in industry and academia, and has been implemented in a variety of programming languages.

How Map Reduce works

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It is designed to work with distributed

systems, where data is spread across multiple machines. Here is how it works:

1. The **Map** function takes an input and divides it into smaller sub-problems, which are then processed by the map tasks in parallel. The output of the map tasks is a set of key-value pairs.
2. The **Reduce** function takes the output of the map tasks and merges the results to produce the final output. The reduce tasks process the key-value pairs in parallel, grouping the values by key and applying the reduce function to each group.
3. The **Master** node is responsible for coordinating the map and reduce tasks. It assigns tasks to the worker nodes and monitors their progress.
4. The **Worker** nodes are responsible for executing the map and reduce tasks assigned to them by the master node.
5. The **Input** and **Output** data is stored in a distributed file system, such as the Hadoop Distributed File System (HDFS), which allows the data to be accessed by the map and reduce tasks.
6. The **Partition** function is used to distribute the output of the map tasks to the reduce tasks. It ensures that all key-value pairs with the same key are processed by the same reduce task.
7. The **Combiner** function is an optional optimization that can be used to reduce the amount of data that needs to be transferred between the map and reduce tasks. It is applied to the output of the map tasks and combines values with the same key before they are sent to the reduce tasks.

MapReduce is a powerful tool for processing large data sets in a distributed environment. It is widely used in big data applications and is the foundation of many data processing frameworks, such as Apache Hadoop.

Developing a Map Reduce application

1. **Understand the problem:** The first step in developing a MapReduce application is to understand the problem you are trying to solve. This involves identifying the input data, the desired output, and the operations that need to be performed on the data to produce the desired output.
2. **Design the Map and Reduce functions:** Once you have a clear understanding of the problem, the next step is to design the Map and Reduce functions. The Map function takes in a key-value pair as input and produces a set of intermediate key-value pairs. The Reduce function takes in an intermediate key and a set of values associated with that key and produces a set of output key-value pairs.
3. **Implement the Map and Reduce functions:** After designing the Map and Reduce functions, the next step is to implement them. This involves

writing code in a programming language such as Java or Python.

4. **Test the application:** Once the Map and Reduce functions have been implemented, the next step is to test the application to ensure that it is producing the desired output. This can be done by running the application on a small dataset and verifying the results.
5. **Optimize the application:** After testing the application, the next step is to optimize it to improve its performance. This can involve tuning the configuration parameters, optimizing the code, or using more efficient data structures and algorithms.
6. **Deploy the application:** Once the application has been optimized, the final step is to deploy it on a Hadoop cluster and run it on the full dataset. This involves configuring the cluster, uploading the input data, and running the application.
7. **Monitor the application:** After the application has been deployed, it is important to monitor its performance to ensure that it is running smoothly. This can involve checking the logs, monitoring the resource usage, and tracking the progress of the application. If any issues are detected, they can be addressed to improve the performance of the application.

Unit Tests with MR Unit

- MRUnit is a library that helps developers to write unit tests for Hadoop MapReduce jobs.
- MRUnit provides a driver class that runs the map or reduce function in isolation and feeds it input data.
- The output of the map or reduce function is captured and can be compared to the expected output.
- MRUnit supports testing of map, reduce, and combiner functions, as well as testing of complete MapReduce jobs.
- MRUnit can be used to test jobs that use the old and the new MapReduce APIs.
- MRUnit tests are written in Java and are run using JUnit.
- MRUnit tests are fast to run because they do not require a Hadoop cluster.
- MRUnit tests can help developers to catch errors early in the development process, before the job is run on a cluster.

Test Data and Local Tests in MapReduce

1. Test data refers to the data used to test the functionality and performance of a MapReduce program. It is important to have a representative sample of data to ensure that the program is working correctly and efficiently.
2. Local tests refer to the practice of testing a MapReduce program on a local machine before deploying it to a cluster. This can help to identify and fix any issues with the program before it is run on a larger scale.

3. One way to generate test data for a MapReduce program is to use a data generation tool. These tools can generate large amounts of data with specific characteristics, such as a certain distribution or data type.
4. Another way to generate test data is to use a subset of real data. This can provide a more realistic test environment, but it may be more difficult to obtain and process the data.
5. When performing local tests, it is important to ensure that the test environment is set up correctly. This includes configuring the local machine to mimic the cluster environment as closely as possible.
6. Local tests can also be used to test the scalability of a MapReduce program. By gradually increasing the size of the test data, it is possible to see how the program performs as the data size increases.
7. It is important to thoroughly test a MapReduce program before deploying it to a cluster. This can help to identify and fix any issues, and ensure that the program runs smoothly and efficiently.

Anatomy of a Map Reduce job run

1. **Input:** The input data is divided into fixed-size pieces called input splits. Each split is processed independently by a separate map task.
2. **Map:** The map function takes key-value pairs as input and produces a set of intermediate key-value pairs as output. The intermediate key-value pairs are sorted by key and partitioned into a number of reduce tasks.
3. **Shuffle:** The shuffle phase transfers the intermediate key-value pairs from the map tasks to the reduce tasks. The reduce tasks sort the incoming data by key.
4. **Reduce:** The reduce function takes the intermediate key-value pairs for a given key and produces a set of final key-value pairs as output. The output of the reduce function is written to the output file system.
5. **Output:** The final output of the MapReduce job is stored in the output file system, typically HDFS.

Failures in MapReduce

MapReduce is a programming model for processing large data sets in parallel across a distributed computing environment. While it is designed to be fault-tolerant, there are still several types of failures that can occur in a MapReduce job. These include:

1. **Task Failure:** A task failure occurs when a map or reduce task fails to complete successfully. This can be due to a variety of reasons, such as a bug in the user code, a hardware failure on the machine running the task, or a network issue. In the case of a task failure, the MapReduce framework will automatically reschedule the task on another machine.

2. **Worker Failure:** A worker failure occurs when a machine running map or reduce tasks fails. This can be due to hardware issues, network issues, or other reasons. In the case of a worker failure, the MapReduce framework will automatically reschedule the tasks that were running on the failed machine to other machines.
3. **Master Failure:** The master node is responsible for coordinating the MapReduce job, including scheduling tasks and monitoring their progress. If the master node fails, the entire MapReduce job will fail. To mitigate this risk, some implementations of MapReduce, such as Hadoop, provide a backup master node that can take over in the case of a master failure.
4. **Network Failure:** A network failure can occur when there is a problem with the network connecting the machines in the distributed computing environment. This can cause tasks to fail, as they are unable to communicate with other tasks or the master node. In the case of a network failure, the MapReduce framework will attempt to reschedule tasks and recover from the failure.
5. **Data Loss:** Data loss can occur when there is a failure in the distributed file system storing the input and output data for the MapReduce job. This can cause tasks to fail, as they are unable to read or write data. In the case of data loss, the MapReduce framework will attempt to recover the lost data and reschedule tasks as necessary.

These are some of the common types of failures that can occur in a MapReduce job. The MapReduce framework is designed to be fault-tolerant and will automatically attempt to recover from failures and complete the job successfully. However, it is still important for users to be aware of these potential failures and design their MapReduce jobs accordingly.

Job Scheduling in MapReduce

Job scheduling in MapReduce is an important aspect of the Hadoop framework. It is responsible for scheduling and monitoring the tasks given by different clients in a Hadoop cluster. The MapReduce scheduling system takes on in six steps:

1. Users submit jobs to a queue, and the cluster runs them in order.
2. The master node distributes Map Tasks and Reduce Tasks to different workers.
3. Map Tasks read the data splits and run the map function on the data which is read in.
4. The output is then sorted and input to reduce tasks.
5. Both job input and output are stored in file systems.
6. Tasks are scheduled and monitored by the framework.

Data locality is one of the most important factors to be considered in order to improve efficiency, as it affects data transmission through the system. There

are different approaches to job scheduling in MapReduce, such as a learning-based scheduler that tries to allocate a task on a node if the incoming task does not affect the tasks already running on that node. This approach tries to find a good mix of jobs for each worker node, and in turn decrease their runtime. The scheduler is made aware of different types of jobs running on the cluster and selects the one that is most compatible with the tasks already running on that node. Machine learning-based solutions can be used to maintain a resource balance on the cluster by not overloading any of the nodes, thereby reducing the overall runtime of the jobs.

Shuffle and Sort in MapReduce

- **Shuffle** is the process of transferring data from the mappers to the reducers in a MapReduce job.
- During the shuffle phase, the MapReduce framework groups the output of the mappers by key and partitions the data to be sent to the reducers.
- The **sort** phase occurs after the shuffle phase and before the reduce phase. During the sort phase, the MapReduce framework sorts the data by key to prepare it for the reduce phase.
- The shuffle and sort phases are important for ensuring that the data is properly organized and distributed to the reducers for processing.
- The MapReduce framework handles the shuffle and sort phases automatically, so the user does not need to write any code for these phases.
- However, the user can customize the partitioning and sorting of the data by implementing a custom partitioner and/or comparator.
- The partitioner determines which reducer a given key-value pair is sent to, while the comparator determines the order in which the key-value pairs are sorted.
- Proper partitioning and sorting of the data can improve the performance and efficiency of a MapReduce job.

Task Execution in MapReduce

MapReduce is a programming model for processing large datasets in parallel across a distributed computing environment. The model consists of two main phases: the Map phase and the Reduce phase.

1. In the Map phase, the input data is divided into chunks and assigned to different worker nodes. Each worker node processes its assigned chunk of data and produces a set of intermediate key-value pairs.
2. These intermediate key-value pairs are then shuffled and sorted by the framework, so that all values associated with the same key are grouped together.
3. In the Reduce phase, the grouped key-value pairs are assigned to different worker nodes. Each worker node processes its assigned key-value pairs and produces the final output.

4. The final output is then collected by the framework and returned to the user.

The MapReduce framework takes care of the details of task scheduling, data distribution, and fault tolerance, allowing the user to focus on writing the Map and Reduce functions. The framework is designed to scale to thousands of machines and to handle failures gracefully.

Map Reduce types in map reduce

MapReduce is a programming model and an associated implementation for processing and generating large data sets. There are two main types of MapReduce jobs: batch and streaming.

1. **Batch MapReduce:** Batch MapReduce jobs are used to process large amounts of data in a single run. These jobs are typically scheduled to run at regular intervals, such as daily or weekly, and are used to perform tasks such as data analysis, data aggregation, and data transformation.
2. **Streaming MapReduce:** Streaming MapReduce jobs are used to process data in real-time as it is generated. These jobs are typically used to perform tasks such as data filtering, data enrichment, and data transformation on data streams.

Both types of MapReduce jobs can be used to process structured, semi-structured, and unstructured data. The choice of which type of MapReduce job to use depends on the specific requirements of the data processing task at hand.

Input Formats in MapReduce

1. InputFormat is responsible for defining the input data for a MapReduce job.
2. It splits the input data into logical chunks called InputSplits and assigns them to individual map tasks for processing.
3. The two most commonly used InputFormats are TextInputFormat and KeyValueTextInputFormat.
4. TextInputFormat reads data line by line and sends each line as a value to the mapper with the key being the byte offset of the line.
5. KeyValueTextInputFormat reads data line by line, but splits each line on a separator character and sends the first part as the key and the second part as the value to the mapper.
6. Other InputFormats include SequenceFileInputFormat, NLineInputFormat, and DBInputFormat.
7. Custom InputFormats can also be created to handle specific data formats.
8. The InputFormat can be specified using the setInputFormatClass method of the Job class.

Output Formats in MapReduce

MapReduce is a programming model for processing large data sets in parallel across a distributed computing environment. The output of a MapReduce job is typically written to the distributed file system, such as Hadoop Distributed File System (HDFS), in a specific output format.

Some common output formats in MapReduce include:

1. **TextOutputFormat:** This is the default output format for MapReduce jobs. It writes data as text files, with each key-value pair separated by a tab character.
2. **SequenceFileOutputFormat:** This output format writes data as binary files in the Hadoop SequenceFile format. SequenceFiles are flat files that store binary key-value pairs and are commonly used for storing intermediate data between MapReduce jobs.
3. **MultipleOutputs:** This class allows writing data to multiple output files, with different output formats for each file. This is useful when the output data needs to be partitioned into multiple files based on certain criteria.
4. **NullOutputFormat:** This output format discards all output data. It is useful when the MapReduce job is used only for its side effects, such as updating a database or generating a report, and the output data is not needed.

It is important to choose the appropriate output format for a MapReduce job, as it can affect the performance and efficiency of the job. The output format can be specified in the job configuration, and custom output formats can also be implemented by extending the `OutputFormat` class.

Map Reduce features

- **Scalability:** MapReduce can process large amounts of data by dividing the work among multiple nodes in a cluster.
- **Fault tolerance:** MapReduce can handle node failures by reassigning tasks to other nodes in the cluster.
- **Data locality:** MapReduce tries to move computation to the data, rather than moving data to the computation, to reduce network traffic and increase performance.
- **Simplicity:** MapReduce provides a simple programming model that abstracts away many of the complexities of distributed computing.
- **Flexibility:** MapReduce can be used to process data in many different formats and can be used for a wide variety of tasks, including data filtering, sorting, and aggregation.
- **Parallelism:** MapReduce can process data in parallel, taking advantage of the multiple cores and processors in modern computers to speed up processing.

Real-world Map Reduce

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It is widely used in the industry for large scale data processing.

1. MapReduce is a framework for processing large datasets in parallel across a large number of machines.
2. The MapReduce model consists of two main phases: the Map phase and the Reduce phase.
3. In the Map phase, the input data is divided into chunks and processed by multiple map tasks in parallel. Each map task processes a chunk of the input data and produces a set of intermediate key-value pairs.
4. In the Reduce phase, the intermediate key-value pairs are shuffled and sorted by key. The reduce tasks then process the key-value pairs with the same key and produce the final output.
5. MapReduce is widely used in the industry for large scale data processing. Some examples of real-world applications of MapReduce include web indexing, data mining, log file analysis, and machine learning.
6. MapReduce can be implemented using various programming languages and platforms, including Java, C++, Python, and Hadoop.
7. The MapReduce model is designed to be scalable, fault-tolerant, and easy to use. It allows developers to focus on the logic of their application, while the framework takes care of the details of parallel processing, data distribution, and fault tolerance.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

- HDFS is a distributed file system designed to run on commodity hardware.
- It is highly fault-tolerant and provides high throughput access to large datasets.
- HDFS is the primary storage system used by Hadoop applications.
- HDFS stores data in blocks, with a default block size of 128 MB.
- Data is replicated across multiple DataNodes for fault tolerance.
- The NameNode manages the file system namespace and regulates access to files by clients.
- The Hadoop environment includes a number of components, including Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop YARN, and Hadoop MapReduce.
- Hadoop Common provides the common utilities and libraries required by other Hadoop modules.
- Hadoop YARN is a resource management platform responsible for managing compute resources in clusters and using them for scheduling of users' applications.
- Hadoop MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster.

HDFS

- HDFS stands for Hadoop Distributed File System.
- It is a distributed file system that handles large data sets running on commodity hardware.
- HDFS is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes.
- It is one of the major components of Apache Hadoop, the others being MapReduce and YARN.
- HDFS is fault-tolerant and designed to be deployed on low-cost, commodity hardware.
- It is the primary storage system used by Hadoop applications.
- This open-source framework works by rapidly transferring data between nodes.
- It is often used by companies who need to handle and store big data.

Design of HDFS

- HDFS stands for Hadoop Distributed File System.
- The design of HDFS was based on the Google File System.
- It was originally built as infrastructure for the Apache Nutch web search engine project but has since become a member of the Hadoop Ecosystem.
- HDFS is a filesystem designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.
- HDFS is designed to reliably store very large files across machines in a large cluster.
- HDFS is designed more for batch processing rather than interactive use by users. The emphasis is on high throughput of data access rather than low latency of data access.
- POSIX imposes many hard requirements that are not needed for applications that are targeted for HDFS.

HDFS Concepts

- HDFS stands for Hadoop Distributed File System.
- It is a distributed file system designed to run on commodity hardware.
- HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware.
- HDFS provides high throughput access to application data and is suitable for applications that have large data sets.
- HDFS stores large files across multiple machines.
- It achieves reliability by replicating the data across multiple hosts.
- HDFS is designed to work with MapReduce, a software framework for distributed processing of large data sets.
- HDFS has a master/slave architecture.
- The HDFS cluster consists of a single NameNode, a master server that manages the file system namespace and regulates access to files by clients.

- In addition, there are a number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on.
- HDFS exposes a file system namespace and allows user data to be stored in files.
- Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.
- The NameNode executes file system namespace operations like opening, closing, and renaming files and directories.
- It also determines the mapping of blocks to DataNodes.
- The DataNodes are responsible for serving read and write requests from the file system's clients.
- The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

Benefits of HDFS

- **Scalability:** HDFS is designed to be highly scalable, allowing for the storage and processing of large amounts of data.
- **Fault Tolerance:** HDFS is designed to be fault-tolerant, meaning that it can continue to operate even in the event of hardware or software failures.
- **Data Reliability:** HDFS ensures data reliability by replicating data across multiple nodes, ensuring that data is not lost in the event of a failure.
- **Cost-Effective:** HDFS is built on top of commodity hardware, making it a cost-effective solution for storing and processing large amounts of data.
- **High Throughput:** HDFS is designed to provide high throughput for large data sets, making it well-suited for big data processing.
- **Easy to Use:** HDFS is easy to use, with a simple and intuitive interface for managing data.
- **Integration with Other Tools:** HDFS can be easily integrated with other big data tools, such as Apache Hadoop, Apache Spark, and Apache Hive, allowing for seamless data processing and analysis.

Challenges of HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets reliably and to stream those data sets at high bandwidth to user applications. Despite its many advantages, there are several challenges associated with HDFS:

1. **Scalability:** As the amount of data stored in HDFS increases, the system must be able to scale to accommodate this growth. This can be challenging, as adding more nodes to the system can increase the complexity of managing the cluster.
2. **Data Integrity:** Ensuring the integrity of data stored in HDFS is crucial. Data corruption can occur due to hardware failures, network issues,

or software bugs. HDFS has built-in mechanisms to detect and recover from data corruption, but these mechanisms can be complex and may not always be effective.

3. **Data Availability:** HDFS is designed to be highly available, but there are still scenarios where data may become unavailable. For example, if a large number of nodes fail simultaneously, data may become temporarily inaccessible. Ensuring high data availability requires careful planning and management of the HDFS cluster.
4. **Performance:** HDFS is designed to provide high throughput for large data sets, but there are still performance challenges that must be addressed. For example, the performance of HDFS can be impacted by network latency, disk I/O, and other factors. Optimizing HDFS performance requires careful tuning and monitoring of the system.
5. **Security:** Securing data stored in HDFS is a critical challenge. HDFS provides several security features, such as authentication and access control, but these features must be properly configured and managed to ensure the security of the data.

These are some of the key challenges associated with HDFS. Addressing these challenges requires careful planning, management, and monitoring of the HDFS cluster.

File Sizes in HDFS

- HDFS is designed to handle very large files.
- The default block size in HDFS is 128 MB, which means that files are split into 128 MB chunks and distributed across the cluster.
- Files smaller than the block size will not be split and will be stored on a single DataNode.
- Files larger than the block size will be split into multiple blocks and distributed across multiple DataNodes.
- The NameNode keeps track of the location of each block and coordinates access to the file.
- HDFS is optimized for streaming large files, so it is not well-suited for handling many small files.
- Storing many small files can lead to inefficient use of disk space and increased load on the NameNode.
- It is recommended to combine small files into larger files before storing them in HDFS. This can be done using tools such as Hadoop Archives (HAR) or Sequence Files.

Block Sizes in HDFS

- The size of the data block in HDFS is 64 MB by default, which can be configured manually .

- In general, the data blocks of size 128MB is used in the industry .
- The size of these HDFS data blocks is 128 MB by default .
- We can configure the block size as per our requirement by changing the `dfs.block.size` property in `hdfs-site.xml` .
- Hadoop distributes these blocks on different slave machines, and the master machine stores the metadata about blocks location .
- The default block size in HDFS was 64mb for Hadoop 1.0 and 128mb for Hadoop 2.0 .
- The block size configuration change can be done on an entire cluster or can be configured for specific blocks .
- The block size and replication factor are configurable per file .
- All blocks in a file except the last block are the same size .
- Users can start a new block without filling out the last block to the configured block size after the support for variable length block was added to `append` and `hsync` .

Block Abstraction in HDFS

- HDFS block size is usually 64MB-128MB .
- Unlike other filesystems, a file smaller than the block size does not occupy the complete block size's worth of memory .
- The block size is kept so large so that less time is made doing disk seeks as compared to the data transfer rate .
- In HDFS the abstraction is made over the blocks of a file rather than a single file which simplifies the storage subsystem .
- Since the size of the blocks is fixed it is easy to manage and calculate how many blocks can be stored on a single disk .
- HDFS exposes a file system namespace and allows user data to be stored in files .
- Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes .
- The NameNode executes file system namespace operations like opening, closing, and renaming files and directories .

Data replication in HDFS

- HDFS stands for Hadoop Distributed File System.
- It is designed to store and manage large amounts of data across multiple machines.
- One of the key features of HDFS is data replication.
- Data replication refers to the process of storing multiple copies of the same data on different machines.
- This is done to ensure data availability and reliability.
- In HDFS, data is automatically replicated across multiple machines.
- The default replication factor is 3, meaning that 3 copies of the data are stored on different machines.

- The replication factor can be configured by the user.
- When a file is uploaded to HDFS, it is split into blocks and each block is replicated across multiple machines.
- If one machine fails, the data can still be accessed from the other machines.
- Data replication in HDFS helps to ensure that data is always available and can be accessed quickly, even in the event of a machine failure.

How does HDFS store

- HDFS stores data in a distributed manner by dividing the data into small pieces and storing it on different DataNodes in the cluster .
- When storing the data, HDFS will divide the files into blocks and store each in the form of DataNode .
- Several DataNodes are connected to the primary node in the cluster, which is known as the NameNode .
- The NameNode executes file system namespace operations like opening, closing, and renaming files and directories .
- The size of each data block is 128MB, which is configurable and can be changed according to your requirement in the hdfs-site.xml file in your Hadoop directory .
- HDFS provides a way for MapReduce to process a subset of large data sets broken into blocks, parallelly on several nodes .
- HDFS is often used by companies who need to handle and store big data .

Read Operations in HDFS

1. HDFS is a distributed file system that stores large data sets across multiple nodes in a cluster.
2. When a client wants to read a file from HDFS, it first contacts the NameNode to get the metadata of the file, including the locations of the blocks that make up the file.
3. The NameNode returns the block locations to the client, which then contacts the DataNodes that store the blocks directly to read the data.
4. The client reads the data from the DataNodes in parallel, taking advantage of the distributed nature of HDFS to achieve high throughput.
5. HDFS also supports data replication, so if a DataNode is unavailable, the client can read the data from a different replica of the block.
6. The client can also specify the preferred location for reading the data, such as reading from a DataNode that is close to the client to reduce network latency.
7. HDFS also supports caching of frequently accessed data to improve read performance.
8. The read performance of HDFS can be further improved by using techniques such as data compression and columnar storage.

Write Operations in HDFS

- HDFS is a distributed file system designed to store large files across multiple machines.
- Write operations in HDFS involve breaking the data into blocks and storing them on different DataNodes in the cluster.
- The NameNode is responsible for managing the metadata of the file system, including the location of the blocks.
- When a client wants to write a file to HDFS, it first contacts the NameNode to obtain a list of DataNodes where the blocks should be stored.
- The client then writes the data to the DataNodes in a pipeline fashion, with the first DataNode in the pipeline storing the first block, the second DataNode storing the second block, and so on.
- Once the data is written to the DataNodes, the client notifies the NameNode, which updates the metadata to reflect the new file.
- The replication factor, which is the number of copies of each block that are stored in the cluster, is configurable and can be set on a per-file basis.
- The default replication factor is three, meaning that each block is stored on three different DataNodes for fault tolerance.
- In the event of a DataNode failure, the NameNode can initiate the replication of the missing blocks to other DataNodes to maintain the desired replication factor.
- HDFS also supports appending data to existing files and truncating files to a specific length.

Java interfaces to HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It provides high-throughput access to application data and is suitable for applications that have large data sets. HDFS is a part of the Apache Hadoop project.

Here are some points on Java interfaces to HDFS:

1. HDFS provides a Java API for interacting with the file system. This API allows developers to perform operations such as creating, reading, writing, and deleting files and directories.
2. The `org.apache.hadoop.fs.FileSystem` class is the primary interface for interacting with HDFS. This class provides methods for performing common file system operations.
3. The `org.apache.hadoop.fs.Path` class represents a file or directory in HDFS. This class is used to specify the location of a file or directory when performing file system operations.
4. The `org.apache.hadoop.conf.Configuration` class is used to configure HDFS. This class allows developers to specify HDFS settings such as the location of the NameNode and the block size.

5. The `org.apache.hadoop.io.IOUtils` class provides utility methods for working with data in HDFS. This class includes methods for copying data between HDFS and local file systems, and for reading and writing data to and from HDFS.
6. The `org.apache.hadoop.fs.FSDataInputStream` and `org.apache.hadoop.fs.FSDataOutputStream` classes provide input and output streams for reading and writing data to and from HDFS. These classes provide a familiar interface for developers who are used to working with Java's `java.io` package.
7. The `org.apache.hadoop.fs.FileStatus` class provides information about a file or directory in HDFS. This class includes information such as the file's length, modification time, and permissions.
8. The `org.apache.hadoop.fs.permission.FsPermission` class represents the permissions of a file or directory in HDFS. This class allows developers to specify the permissions of a file or directory when creating it, or to change the permissions of an existing file or directory.

Command Line Interface to HDFS

- HDFS, or Hadoop Distributed File System, is a distributed file system designed to run on commodity hardware.
- The command line interface (CLI) to HDFS is a way to interact with the file system using commands entered in a terminal or command prompt.
- Some common commands for interacting with HDFS include:
 - `hadoop fs -ls`: lists the contents of a directory in HDFS
 - `hadoop fs -mkdir`: creates a new directory in HDFS
 - `hadoop fs -put`: copies a file from the local file system to HDFS
 - `hadoop fs -get`: copies a file from HDFS to the local file system
 - `hadoop fs -rm`: deletes a file or directory in HDFS
- These commands can be used to perform basic file system operations such as creating, deleting, and moving files and directories.
- The CLI can also be used to perform more advanced operations such as setting permissions and replication factors for files and directories.
- To use the CLI, the Hadoop client must be installed and configured on the local machine.
- The CLI provides a powerful and flexible way to interact with HDFS, making it a valuable tool for managing and working with data stored in the file system.

Hadoop file system interfaces

- Hadoop has a variety of file systems that can be implemented concretely.
- The Java abstract class `org.apache.hadoop.fs.FileSystem` represents a file system in Hadoop.
- Hadoop provides numerous interfaces to its various filesystems, and it generally uses the URI scheme to choose the right filesystem instance to

communicate with.

- Hadoop is capable of running various file systems and HDFS is just one single implementation.
- Hadoop provides a command interface to interact with HDFS.
- The built-in servers of namenode and datanode help users to easily check the status of the cluster.
- Hadoop is written in Java, so most Hadoop filesystem interactions are mediated through the Java API.
- The `org.apache.hadoop.fs.FileSystem` class implements several interfaces, including `Closeable`, `AutoCloseable`, `Configurable`, `org.apache.hadoop.fs.PathCapabilities`, and `org.apache.hadoop.security.token.DelegationToken`.
- There are several direct known subclasses of `org.apache.hadoop.fs.FileSystem`, including `AdlFileSystem`, `FilterFileSystem`, `FTPFileSystem`, `NativeAzureFileSystem`, `NativeS3FileSystem`, `RawLocalFileSystem`, and `ViewFileSystem`.

Data flow in HDFS

HDFS (Hadoop Distributed File System) is a distributed file system that handles large data sets running on commodity hardware. It is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes.

- **Reading data:** As data is stored in a distributed manner, reading operations will run in parallel. The client interacts directly with the slaves and reads data blocks from there. The client opens the file it wishes to read by calling `open()` on the File System Object (which for HDFS is an instance of Distributed File System).
- **Writing data:** The client creates the file by calling `create()` on `DistributedFileSystem` (DFS). DFS makes an RPC call to the name node to create a new file in the file system's namespace, with no blocks associated with it. As the client writes data, the `DFSOutputStream` splits it into packets, which it writes to an internal queue called the data queue.
- **Integration with other systems:** HDFS can be integrated with other systems, such as SQL Server Integration Services (SSIS), to write data to an HDFS file. The supported file formats are Text, Avro, and ORC.

Data Ingest with Flume and Sqoop in HDFS

1. **Flume** is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data.
2. It has a simple and flexible architecture based on streaming data flows.
3. It is robust and fault-tolerant with tunable reliability mechanisms and many failover and recovery mechanisms.
4. Flume can be used to ingest data from various sources into HDFS.
5. **Sqoop** is a tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases.

6. Sqoop can be used to import data from external structured datastores into HDFS or related storage systems.
7. Both Flume and Sqoop can be used to ingest data into HDFS, but they serve different purposes and have different strengths.
8. Flume is more suited for ingesting streaming data or data that is continuously generated, while Sqoop is more suited for ingesting data from structured datastores in bulk.

Hadoop archives in HDFS

- Hadoop archives (HAR) are a file archiving facility provided by Hadoop Distributed File System (HDFS).
- HAR files are used to compact small files in HDFS into larger files to reduce the load on the NameNode.
- HAR files are created using the `hadoop archive` command.
- The `hadoop archive` command takes a list of files and directories as input and creates a single HAR file as output.
- The HAR file is stored in HDFS and can be accessed using the `har://` URI scheme.
- HAR files can be used to improve the performance of MapReduce jobs by reducing the number of small files that need to be processed.
- HAR files can be unpacked using the `hadoop fs -cp` command.
- HAR files are read-only and cannot be modified once created.
- HAR files can be deleted using the `hadoop fs -rm` command.

Hadoop I/O

- Hadoop comes with a set of primitives for data I/O.
- Some of these techniques are more general than Hadoop, such as data integrity and compression, but deserve special consideration when dealing with multi-terabyte datasets.
- Hadoop has its own compact and fast serialization format, Writables, that MapReduce programs use to generate keys and value types.
- Hadoop's own Record I/O (found in the `org.apache.hadoop.record` package) has an IDL that is compiled into Writable objects, which makes it convenient for generating types that are compatible with MapReduce.
- However, Record I/O was not widely used, and has been deprecated in favor of Avro.

Compression in Hadoop IO

1. Hadoop provides support for various compression codecs and formats, such as gzip, bzip2, snappy, and lz4.
2. Compression can be used to reduce the size of data stored on HDFS, which can save storage space and reduce the amount of data that needs to be transferred between nodes during processing.

3. Compression can be applied at different stages of data processing, such as during input, output, or intermediate stages.
4. Hadoop provides a configurable and extensible framework for adding new compression codecs and formats.
5. Compression can be enabled or disabled for individual files or for entire directories.
6. Compression can be configured at the cluster level or at the job level.
7. Compression can have a significant impact on the performance of data processing, and the choice of compression codec and format should be carefully considered based on the characteristics of the data and the processing requirements.
8. Some compression codecs, such as gzip and bzip2, provide a high level of compression but can be computationally expensive, while others, such as snappy and lz4, provide faster compression and decompression but with a lower level of compression.
9. Hadoop provides tools for estimating the compression ratio and performance of different codecs and formats on sample data.
10. Hadoop also provides support for splittable compression formats, which allow data to be processed in parallel even when compressed. This can improve the performance of data processing by allowing multiple tasks to work on the same compressed file simultaneously. However, not all compression formats are splittable, and the choice of compression format should take this into consideration.

Serialization in Hadoop IO

1. Serialization is the process of converting data structures or objects into a binary format that can be stored or transmitted across a network.
2. In Hadoop, serialization is used to store data in HDFS and to transfer data between nodes during MapReduce jobs.
3. Hadoop provides its own serialization framework called Writable, which is optimized for performance and is used by default in MapReduce jobs.
4. Writable classes implement the Writable interface, which defines two methods: `write(DataOutput out)` and `readFields(DataInput in)`.
5. The `write` method is used to serialize the object, while the `readFields` method is used to deserialize the object.
6. Hadoop also supports other serialization frameworks such as Avro, Protocol Buffers, and Thrift, which can be used by implementing custom `InputFormat` and `OutputFormat` classes.
7. These alternative serialization frameworks provide additional features such as schema evolution and language independence, but may not be as performant as the native Writable framework.
8. Choosing the appropriate serialization framework depends on the specific requirements of the application, such as performance, data size, and data complexity.

Avro and file based data structures in Hadoop io

- Avro is a data serialization system that is used to encode data structures into a compact binary format.
- Avro is used in Hadoop to store and process large amounts of data in a distributed computing environment.
- Avro provides a way to define data structures using a schema, which can be used to validate and encode data.
- Avro schemas are defined using JSON, which makes them easy to read and write.
- Avro supports schema evolution, which means that data can be read even if the schema has changed.
- File-based data structures in Hadoop are used to store and organize data on disk.
- Hadoop provides several file-based data structures, including SequenceFile, MapFile, and SetFile.
- SequenceFile is a flat file that stores key-value pairs in a binary format.
- MapFile is an indexed file that provides fast lookup of key-value pairs.
- SetFile is a file that stores a set of keys, with no associated values.
- These file-based data structures are used to store intermediate data during MapReduce processing, as well as to store the final output of a MapReduce job.
- Hadoop provides APIs for reading and writing these file-based data structures, making it easy to work with data stored in Hadoop.

Hadoop Environment

- Hadoop is an open-source software framework for storing and processing large datasets.
- It is designed to scale up from a single server to thousands of machines, each offering local computation and storage.
- Hadoop is based on the MapReduce programming model, which allows for the distributed processing of large data sets across clusters of computers.
- Hadoop consists of two main components: the Hadoop Distributed File System (HDFS) and the MapReduce engine.
- HDFS is a distributed file system that provides high-throughput access to application data.
- The MapReduce engine is responsible for scheduling and coordinating the processing of data across the cluster.
- Hadoop can be installed on commodity hardware, making it a cost-effective solution for managing large datasets.
- Hadoop is commonly used for data-intensive applications such as data mining, log analysis, and machine learning.
- Hadoop can be run in a variety of environments, including on-premises, in the cloud, or in a hybrid setup.
- To set up a Hadoop environment, you will need to install and configure the

Hadoop software, as well as set up the necessary hardware and network infrastructure.

- There are several tools and frameworks available to help with the setup and management of a Hadoop environment, including Apache Ambari, Cloudera Manager, and Hortonworks Data Platform.

Setting up a Hadoop cluster in Hadoop Environment

1. **Install Java:** Hadoop requires Java to be installed on all the nodes in the cluster. Make sure that the same version of Java is installed on all the nodes.
2. **Install Hadoop:** Download and install Hadoop on all the nodes in the cluster. Make sure that the same version of Hadoop is installed on all the nodes.
3. **Configure Hadoop:** Edit the Hadoop configuration files on all the nodes to set up the cluster. This includes setting the hostname and port for the NameNode, DataNodes, and other services.
4. **Format the NameNode:** On the NameNode, run the command `hdfs namenode -format` to format the Hadoop Distributed File System (HDFS).
5. **Start the Hadoop services:** On all the nodes, start the Hadoop services such as the NameNode, DataNode, ResourceManager, and NodeManager.
6. **Verify the cluster:** Run the command `hdfs dfsadmin -report` on the NameNode to verify that the cluster is set up correctly and all the DataNodes are connected to the NameNode.
7. **Create directories:** Create the necessary directories in HDFS for storing data and running MapReduce jobs.
8. **Test the cluster:** Run a sample MapReduce job to test that the cluster is set up correctly and is functioning as expected.

Cluster Specification in Hadoop Environment

- A Hadoop cluster is a collection of computers, known as nodes, that are networked together to perform data storage and processing tasks.
- The nodes in a Hadoop cluster are divided into two types: master nodes and worker nodes.
- The master nodes run the Hadoop Distributed File System (HDFS) NameNode and the YARN ResourceManager services. These services are responsible for managing the storage and processing resources of the cluster.
- The worker nodes run the HDFS DataNode and YARN NodeManager services. These services are responsible for storing data and executing processing tasks.

- The number of nodes in a Hadoop cluster can vary depending on the amount of data to be stored and processed, and the processing power required.
- The hardware specifications of the nodes in a Hadoop cluster should be chosen based on the workload requirements. For example, a cluster that will be used for processing large amounts of data may require nodes with more storage capacity and processing power.
- The network infrastructure of a Hadoop cluster should be designed to provide high bandwidth and low latency connectivity between the nodes. This is important for efficient data transfer and processing.
- The Hadoop software stack should be installed and configured on all nodes in the cluster. This includes the Hadoop core components, as well as any additional tools and libraries required for the intended workload.
- The Hadoop cluster should be monitored and managed to ensure that it is operating efficiently and effectively. This includes monitoring the performance and health of the nodes, and performing maintenance tasks such as software updates and data backups.

Cluster Setup and Installation in Hadoop Environment

1. **Prerequisites:** Before setting up a Hadoop cluster, ensure that all the machines in the cluster meet the hardware and software requirements. The machines should have a compatible operating system, Java Development Kit (JDK), and Secure Shell (SSH) installed.
2. **Download and Install Hadoop:** Download the latest stable release of Hadoop from the Apache Hadoop website. Extract the downloaded file and move it to the desired location. Set the environment variables for Hadoop by adding the Hadoop bin directory to the PATH variable.
3. **Configure Hadoop:** Hadoop configuration files are located in the \$HADOOP_HOME/etc/hadoop directory. Edit the core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml files to configure the Hadoop cluster.
4. **Set up SSH:** Hadoop requires password-less SSH access between the machines in the cluster. Generate an SSH key pair on the master machine and copy the public key to all the slave machines.
5. **Format the Hadoop File System:** Before starting the Hadoop cluster, format the Hadoop Distributed File System (HDFS) by running the `hdfs namenode -format` command on the master machine.
6. **Start the Hadoop Cluster:** Start the Hadoop daemons by running the `start-dfs.sh` and `start-yarn.sh` scripts on the master machine. Verify that the Hadoop cluster is running by checking the web interface of the NameNode and ResourceManager.
7. **Add Data to HDFS:** Use the `hdfs dfs -put` command to add data

to HDFS. The data can be processed using MapReduce or other Hadoop ecosystem tools.

8. **Monitor the Hadoop Cluster:** Monitor the Hadoop cluster using the web interface of the NameNode and ResourceManager. The logs of the Hadoop daemons can also provide useful information for troubleshooting.

Hadoop Configuration in Hadoop Environment

Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers. In order to use Hadoop effectively, it is important to properly configure the environment in which it will be running. Here are some key points to consider when configuring Hadoop in a Hadoop environment:

1. **Hardware Configuration:** The hardware on which Hadoop will be running should be properly configured to meet the demands of the system. This includes ensuring that there is enough memory, storage, and processing power to handle the data being processed.
2. **Network Configuration:** Hadoop relies on a network of computers to distribute data and processing tasks. It is important to ensure that the network is properly configured to allow for efficient communication between the nodes in the cluster.
3. **Hadoop Configuration Files:** Hadoop has several configuration files that control various aspects of the system. These files should be properly configured to reflect the specific needs of the environment in which Hadoop is running.
4. **Data Storage:** Hadoop stores data in a distributed file system called HDFS. It is important to properly configure the data storage to ensure that data is stored efficiently and can be accessed quickly.
5. **Resource Management:** Hadoop uses a resource manager called YARN to allocate resources to different processing tasks. Properly configuring YARN can help to ensure that resources are used efficiently and that processing tasks are completed in a timely manner.

Overall, properly configuring Hadoop in a Hadoop environment is essential for ensuring that the system runs efficiently and effectively. By considering the above points, you can help to ensure that your Hadoop environment is properly configured to meet your needs.

Security in Hadoop in Hadoop Environment

Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers. As with any system that stores and processes data, security is an important consideration in a

Hadoop environment. Here are some key points to consider when it comes to security in Hadoop:

1. **Authentication:** Authentication is the process of verifying the identity of a user or system. In a Hadoop environment, this can be achieved through the use of Kerberos, a network authentication protocol that uses strong cryptography to verify the identity of users and services.
2. **Authorization:** Authorization is the process of determining what actions a user or system is allowed to perform. In a Hadoop environment, this can be achieved through the use of file and directory permissions, as well as through the use of Access Control Lists (ACLs) and other security mechanisms.
3. **Data Protection:** Data protection refers to the measures taken to ensure the confidentiality, integrity, and availability of data. In a Hadoop environment, this can be achieved through the use of encryption, both for data at rest and data in transit, as well as through the use of secure data transfer protocols.
4. **Auditing:** Auditing is the process of recording and reviewing system activity to detect and investigate security incidents. In a Hadoop environment, this can be achieved through the use of log files and other monitoring tools.
5. **Network Security:** Network security refers to the measures taken to protect the network infrastructure that supports a Hadoop environment. This can include the use of firewalls, intrusion detection and prevention systems, and other security mechanisms.

In summary, security in a Hadoop environment involves a combination of authentication, authorization, data protection, auditing, and network security measures. By implementing these measures, it is possible to ensure the security of data and systems in a Hadoop environment.

Administering Hadoop in Hadoop Environment

1. **Installation and Configuration:** The first step in administering Hadoop in a Hadoop environment is to install and configure the software. This includes setting up the Hadoop Distributed File System (HDFS) and the MapReduce framework.
2. **Cluster Management:** Once Hadoop is installed and configured, the next step is to manage the cluster. This includes adding and removing nodes, monitoring the health of the cluster, and balancing data across the nodes.
3. **Data Management:** Data management is another important aspect of administering Hadoop in a Hadoop environment. This includes ingesting

data into HDFS, managing the data lifecycle, and ensuring data availability.

4. **Security:** Security is a critical aspect of administering Hadoop in a Hadoop environment. This includes setting up authentication and authorization, configuring data encryption, and monitoring for security threats.
5. **Performance Tuning:** Performance tuning is an ongoing process in administering Hadoop in a Hadoop environment. This includes optimizing the configuration of Hadoop and the underlying hardware, as well as monitoring and tuning the performance of the cluster.
6. **Backup and Recovery:** Backup and recovery is an important aspect of administering Hadoop in a Hadoop environment. This includes setting up a backup strategy, performing regular backups, and ensuring that data can be recovered in the event of a failure.
7. **Upgrades and Maintenance:** Upgrades and maintenance are an ongoing part of administering Hadoop in a Hadoop environment. This includes applying patches and updates, upgrading to new versions of Hadoop, and performing regular maintenance tasks.

In summary, administering Hadoop in a Hadoop environment involves a wide range of tasks, including installation and configuration, cluster management, data management, security, performance tuning, backup and recovery, and upgrades and maintenance. These tasks require a deep understanding of Hadoop and its underlying technologies, as well as the ability to monitor and manage a complex distributed system.

HDFS monitoring & maintenance in Hadoop Environment

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

Monitoring and maintenance of HDFS in a Hadoop environment is important to ensure the smooth functioning of the system. Here are some points to consider:

1. **Monitoring disk usage:** It is important to monitor the disk usage of the Hadoop cluster to ensure that there is enough space available for data storage. This can be done using the `hdfs dfsadmin -report` command, which provides information about the capacity, used and remaining space on each data node.
2. **Checking data node status:** The status of the data nodes in the Hadoop cluster can be checked using the `hdfs dfsadmin -report` command. This command provides information about the number of live and dead data nodes, as well as the number of decommissioned data nodes.

3. **Monitoring data transfer:** Data transfer between the data nodes and the name node can be monitored using the `hdfs dfsadmin -top` command. This command provides information about the top users of the HDFS cluster, as well as the top data transfer operations.
4. **Checking for corrupt blocks:** Corrupt blocks in the Hadoop cluster can be checked using the `hdfs fsck /` command. This command checks the health of the Hadoop file system and reports any corrupt or missing blocks.
5. **Performing regular backups:** Regular backups of the Hadoop cluster should be performed to ensure that data is not lost in case of a failure. This can be done using the `hdfs dfsadmin -safemode enter` and `hdfs dfsadmin -saveNamespace` commands.
6. **Performing regular maintenance:** Regular maintenance of the Hadoop cluster, such as adding or removing data nodes, should be performed to ensure the smooth functioning of the system. This can be done using the `hdfs dfsadmin -refreshNodes` command.

In conclusion, monitoring and maintenance of HDFS in a Hadoop environment is important to ensure the smooth functioning of the system. Regular monitoring of disk usage, data node status, data transfer, and corrupt blocks, as well as performing regular backups and maintenance, can help to keep the system running smoothly.

Hadoop Benchmarks in Hadoop Environment

Hadoop benchmarks are used to measure the performance of a Hadoop cluster. These benchmarks can help identify bottlenecks and optimize the performance of the cluster. Some common Hadoop benchmarks include:

1. **TeraSort:** This benchmark measures the time it takes to sort a large dataset using the Hadoop MapReduce framework.
2. **TestDFSIO:** This benchmark measures the read and write performance of the Hadoop Distributed File System (HDFS).
3. **Nutch indexing:** This benchmark measures the time it takes to index a large dataset using the Apache Nutch web crawler.
4. **PageRank:** This benchmark measures the time it takes to compute the PageRank of a large dataset using the Hadoop MapReduce framework.

These benchmarks can be run on a Hadoop cluster to measure its performance and identify areas for improvement. It is important to note that the results of these benchmarks can vary depending on the hardware and configuration of the cluster, so it is important to use them as a guide rather than an absolute measure of performance.

Hadoop in the cloud in Hadoop Environment

- Hadoop is an open-source software framework for storing and processing large datasets.
- Hadoop can be deployed in a variety of environments, including on-premises, in the cloud, or in a hybrid configuration.
- Cloud-based Hadoop deployments offer several advantages over on-premises deployments, including scalability, flexibility, and cost-effectiveness.
- Cloud providers such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform offer managed Hadoop services that can simplify the deployment and management of Hadoop clusters.
- When deploying Hadoop in the cloud, it is important to consider factors such as data locality, security, and cost.
- Data locality refers to the proximity of the data to the compute resources that will process it. In a cloud-based Hadoop deployment, data may need to be transferred between the cloud storage and the Hadoop cluster, which can impact performance.
- Security is another important consideration when deploying Hadoop in the cloud. Cloud providers offer a range of security features and best practices to help secure Hadoop data and clusters.
- Cost is also an important factor to consider when deploying Hadoop in the cloud. Cloud-based Hadoop deployments can offer cost savings compared to on-premises deployments, but it is important to carefully evaluate and manage costs to ensure that the deployment remains cost-effective.

Unit 3 - Hadoop Eco System and YARN, NoSQL databases, MongoDB, Spark, Scala

- **Hadoop Eco System:** Hadoop is an open-source software framework for storing and processing large datasets. The Hadoop ecosystem includes a variety of tools and technologies that can be used to work with Hadoop, including Hadoop Distributed File System (HDFS), MapReduce, Pig, Hive, HBase, and others.
- **YARN:** YARN (Yet Another Resource Negotiator) is a resource management layer in Hadoop that allows multiple data processing engines to share resources in a cluster. It is responsible for managing and allocating resources to applications running on the Hadoop cluster.
- **NoSQL databases:** NoSQL databases are non-relational databases that are designed to handle large amounts of unstructured or semi-structured data. They are often used in big data and real-time web applications. Some popular NoSQL databases include MongoDB, Cassandra, and Redis.
- **MongoDB:** MongoDB is a document-based NoSQL database that stores data in flexible, JSON-like documents. It is designed to be scalable, high-performance, and easy to use.
- **Spark:** Apache Spark is an open-source, distributed computing system

that is designed to process large amounts of data quickly. It can be used for a variety of tasks, including data processing, machine learning, and graph processing.

- **Scala:** Scala is a programming language that combines object-oriented and functional programming. It is often used with Apache Spark to write scalable and efficient data processing code.

Hadoop Eco System and YARN

1. **Hadoop** is an open-source software framework for storing and processing large datasets using distributed computing.
2. The **Hadoop ecosystem** refers to the various components and tools that work with Hadoop to enhance its capabilities and performance.
3. One of the key components of the Hadoop ecosystem is **YARN** (Yet Another Resource Negotiator), which is responsible for managing resources and scheduling tasks across the Hadoop cluster.
4. YARN separates the resource management and job scheduling functions from the data processing component, allowing for more efficient and scalable processing of large datasets.
5. YARN consists of two main components: the **Resource Manager** and the **Node Manager**.
6. The **Resource Manager** is responsible for managing the allocation of resources across the cluster, while the **Node Manager** runs on each node in the cluster and is responsible for managing the resources and tasks on that node.
7. YARN also includes an **Application Master** for each application, which is responsible for coordinating the execution of tasks for that application and communicating with the Resource Manager to request resources.
8. YARN supports multiple data processing frameworks, including **MapReduce**, **Spark**, and **Tez**, allowing users to choose the best tool for their specific needs.
9. The Hadoop ecosystem also includes a variety of other tools and components, such as **HDFS** (Hadoop Distributed File System), **Hive**, **Pig**, and **Sqoop**, which provide additional functionality for data storage, processing, and analysis.
10. Overall, the Hadoop ecosystem and YARN provide a powerful and flexible platform for managing and processing large datasets, making it a popular choice for big data applications.

Hadoop ecosystem components

The Hadoop ecosystem is made up of several components that work together to provide a complete big data solution. These components include:

1. **HDFS (Hadoop Distributed File System):** This is the backbone of Hadoop, which runs on Java language and stores data in Hadoop applica-

tions. It acts as a command interface to interact with Hadoop. The two components of HDFS are Data node and Name Node .

2. **HBASE:** It is an open-source framework storing all types of data and doesn't support the SQL database .
3. **YARN:** Yet Another Resource Negotiator (YARN) is a resource management layer in Hadoop .
4. **Sqoop:** Sqoop is a tool designed for efficiently transferring bulk data between Apache Hadoop and structured data stores such as relational databases .
5. **Apache Spark:** Apache Spark is an open-source distributed general-purpose cluster-computing framework .
6. **Apache Flume:** Apache Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data .
7. **Hadoop MapReduce:** Hadoop MapReduce is a software framework for easily writing applications which process vast amounts of data (multi-terabyte data-sets) in-parallel on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner .
8. **Apache Pig:** Apache Pig is a platform for analyzing large data sets that consists of a high-level language for expressing data analysis programs, coupled with infrastructure for evaluating these programs .

These are some of the main components of the Hadoop ecosystem, but there are many more tools and projects that can be used in conjunction with Hadoop to provide a complete big data solution .

Schedulers in Hadoop Ecosystem

Schedulers in Hadoop are general purpose systems that allow the system to perform high level performance processing of data on distributed node sets known as Hadoop. They also help in ensuring optimum utilization of resources and access to unused levels of capacity .

There are mainly 3 types of Schedulers in Hadoop :

1. **FIFO (First In First Out) Scheduler:** This scheduler schedules tasks in the order they are received.
2. **Capacity Scheduler:** This scheduler is designed to allow multiple tenants to share a large cluster while maximizing the utilization of the resources.
3. **Fair Scheduler:** This scheduler assigns resources to jobs in a way that all jobs get, on average, an equal share of resources over time.

These Schedulers are actually a kind of algorithm that is used to schedule tasks in a Hadoop cluster when requests are received from different clients .

Fair and Capacity in Hadoop Ecosystem

- The Hadoop YARN scheduler is responsible for assigning resources to the applications submitted by users. In Hadoop YARN, these can be any YARN application in addition to MapReduce applications .
- Fair Scheduler allows YARN applications to justly share resources in large Hadoop clusters. With this scheduler, there is no need for reserving a set amount of capacity because it will dynamically balance resources between all running applications .
- The central idea is that the available resources in the Hadoop Map-Reduce cluster are partitioned among multiple organizations who collectively fund the cluster based on computing needs. There is an added benefit that an organization can access any excess capacity not being used by others .
- There are mainly 3 types of Schedulers in Hadoop: FIFO (First In First Out) Scheduler, Capacity Scheduler, and Fair Scheduler. These Schedulers are actually a kind of algorithm that we use to schedule tasks in a Hadoop cluster when we receive requests from different-different clients .

Hadoop 2.0 New Features - NameNode high availability

- High Availability was a new feature added to Hadoop 2.x to solve the Single point of failure problem in the older versions of Hadoop.
- Hadoop 2.0 overcomes this SPOF shortcoming by providing support for multiple NameNodes.
- It introduces Hadoop 2.0 High Availability feature that brings in an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture which is configured for automatic failover.
- The main motive of the Hadoop 2.0 High Availability project is to render availability to large data applications 24/7 through the deployment of 2 Hadoop Name Nodes.
- The HDFS NameNode High Availability feature enables you to run redundant NameNodes in the same cluster in an Active/Passive configuration with a hot standby.
- This eliminates the NameNode as a potential single point of failure (SPOF) in an HDFS cluster.

HDFS Federation in Hadoop Ecosystem

- HDFS Federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture.
- It overcomes HDFS architecture limitations by adding multiple NameNode/namespaces support to HDFS, allowing the use of more than one NameNode/namespace .

- This overcomes the isolation, scalability, and performance limitations of the prior HDFS architecture and opens up the architecture for future innovations.
- The HDFS Federation architecture has a collection of Namespace volumes, which are self-contained management units. On deleting the NameNode or namespace, the corresponding block pool present in the DataNodes also gets deleted. On upgrading the cluster, each namespace volume gets upgraded as a unit.
- HDFS has two main layers: Namespace and Block Storage Service. Namespace consists of directories, files, and blocks and supports all the namespace-related file system operations such as create, delete, modify and list files and directories. Block Storage Service has two parts: Block Management (performed in the Namenode) and Block Storage (performed in the DataNodes).

MRv2 in Hadoop ecosystem

- MRv2, also known as Hadoop 2, is a version of Hadoop where the resource management and scheduling tasks are separated from MapReduce by YARN (Yet Another Resource Negotiator) .
- In Hadoop 2, the resource management and scheduling layer lies beneath the MapReduce layer .
- MRv2 is an application framework that runs within YARN .
- In Hadoop 1, all DataNodes are dedicated to Map and Reduce tasks and cannot be used for other processing. In Hadoop 2, the cluster's capacity is measured in MapReduce slots .
- In Hadoop version 1, MapReduce was responsible for both processing and cluster resource management. In Apache Hadoop version 2, cluster resource management has been moved from MapReduce into YARN, thus enabling other application engines to utilize YARN and Hadoop, while also improving the performance of MapReduce .
- YARN is very important to know where exactly it fits in Hadoop 2 now. Hadoop 2 has undergone a complete change in terms of architecture and components compared to Hadoop 1 .

YARN

- Yarn is a long continuous length of interlocked fibers, used in sewing, crocheting, knitting, weaving, embroidery, ropemaking, and the production of textiles .
- Thread is a type of yarn intended for sewing by hand or machine .
- Yarn comes in an amazing variety of types and styles, allowing you to personalize your project with color and texture by choosing a yarn that fits your style .
- If you find a knit or crochet pattern you like, look at the yarn it recommends .

Running MRv1 in YARN

- MRv1, also known as MapReduce version 1, is a framework for processing large data sets in a distributed computing environment.
- YARN, or Yet Another Resource Negotiator, is a resource management layer in Hadoop that allows multiple data processing engines to share resources in a cluster.
- To run MRv1 in YARN, the following steps can be followed:
 1. Install and configure Hadoop with YARN.
 2. Set up the MRv1 job configuration, including input and output paths, mapper and reducer classes, and any other necessary parameters.
 3. Submit the MRv1 job to the YARN resource manager using the `hadoop jar` command.
 4. Monitor the progress of the job using the YARN web UI or command line tools.
 5. Once the job is complete, retrieve the results from the specified output path.
- Running MRv1 in YARN allows for efficient resource utilization and improved scalability compared to running MRv1 without YARN.

NoSQL Databases

- NoSQL stands for “non-SQL” or “non-relational” database, which provides a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in relational databases.
- The data structures used by NoSQL databases, such as key-value pair, wide column, graph, or document, are different from those used by default in relational databases, making some operations faster in NoSQL.
- NoSQL databases are designed to be used across large distributed systems and are much more scalable and faster at handling very large data loads than traditional relational databases.
- NoSQL databases come in a variety of types based on their data model, including document, key-value, wide-column, and graph.
- They provide flexible schemas and scale easily with large amounts of data and high user loads.
- Some popular NoSQL databases include Apache CouchDB, Elasticsearch, and Couchbase.

Introduction to NoSQL databases

- NoSQL databases are non-relational databases that store and retrieve data in ways that do not involve the use of a fixed schema like traditional relational databases.
- NoSQL databases are designed to handle large volumes of structured, semi-structured, or unstructured data.
- They are often used in big data and real-time web applications.

- NoSQL databases can be classified into four main types: document-based, key-value, column-family, and graph databases.
- Some popular NoSQL databases include MongoDB, Cassandra, Redis, and Neo4j.
- NoSQL databases offer several advantages over traditional relational databases, including scalability, flexibility, and performance.
- However, they may not be suitable for all use cases and may require a different approach to data modeling and querying.

MongoDB

- MongoDB is an open-source, non-relational database management system (DBMS) that uses flexible documents instead of tables and rows to process and store various forms of data.
- It is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built-in and easy to use.
- MongoDB is free to use. Versions released prior to October 16, 2018, are published under the AGPL.
- It is a document-oriented database, which means that data is stored as documents, and documents are grouped in collections. The document model is a lot more natural for developers to work with because documents are self-contained and can be treated as objects.
- MongoDB helps remove barriers to innovation for organizations in any industry.
- The Community version of our distributed database offers a flexible document data model along with support for ad-hoc queries, secondary indexing, and real-time aggregations to provide powerful ways to access and analyze your data.
- MongoDB is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program, MongoDB uses JSON-like documents with optional schemas. MongoDB is developed by MongoDB Inc. and licensed under the Server Side Public License (SSPL) which is deemed non-free by several distributions.

Introduction to MongoDB

MongoDB is a cross-platform document-oriented database program. It is classified as a NoSQL database program, which means it does not use the traditional tabular relational database structure. Instead, it uses JSON-like documents with optional schemas. Some key features of MongoDB include:

- **Document-based:** Data is stored in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.
- **Scalable:** MongoDB is horizontally scalable, meaning it can handle large amounts of data by spreading it across multiple servers.
- **Flexible:** MongoDB's dynamic schema allows for easy modification of data structure without downtime.
- **Powerful querying:** MongoDB supports a wide range of querying

options, including ad-hoc queries, indexing, and real-time aggregation. - **High availability:** MongoDB provides high availability through built-in replication and automatic failover. - **Expressive language:** MongoDB's query language is expressive and allows for powerful and flexible data manipulation.

MongoDB is widely used for web and mobile applications, real-time analytics, content management, and many other use cases. It is a popular choice for modern application development due to its flexibility, scalability, and ease of use.

Data Types in MongoDB

MongoDB supports several data types, including:

1. **String:** This is the most commonly used data type to store data. Strings in MongoDB must be UTF-8 valid.
2. **Integer:** This type is used to store a numerical value. Integer can be a 32-bit or 64-bit, depending on the server architecture.
3. **Boolean:** This type is used to store a boolean (true/ false) value.
4. **Double:** This type is used to store floating-point values.
5. **Min/ Max keys:** This type is used to compare a value against the lowest and highest BSON elements, respectively.
6. **Arrays:** This type is used to store arrays or lists.
7. **Timestamp:** This type is used to store a timestamp. This can be handy for recording when a document has been modified or added.
8. **Object:** This type is used to store embedded documents.
9. **Null:** This type is used to store a Null value.
10. **Symbol:** This type is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
11. **Date:** This type is used to store the current date or time in UNIX time format. You can specify your own date time by creating an object of Date and passing the date time string as a parameter.
12. **Object ID:** This type is used to store the document's ID.
13. **Binary data:** This type is used to store binary data.
14. **Code:** This type is used to store JavaScript code into the document.
15. **Regular expression:** This type is used to store regular expression.

These are the most commonly used data types in MongoDB. Each data type has its own specific use case and it is important to choose the right data type for the data being stored.

Creating Documents in MongoDB

1. **Introduction:** MongoDB is a NoSQL database that stores data in the form of documents. These documents are stored in collections, which are similar to tables in a relational database.
2. **Creating a Document:** To create a document in MongoDB, you can use

the `insertOne()` or `insertMany()` methods. The `insertOne()` method is used to insert a single document into a collection, while the `insertMany()` method is used to insert multiple documents into a collection at once.

3. **Syntax:** The syntax for the `insertOne()` method is as follows:

```
db.collection.insertOne(
  <document>,
  {
    writeConcern: <document>,
    ordered: <boolean>
  }
)
```

The syntax for the `insertMany()` method is as follows:

```
db.collection.insertMany(
  [ <document1>, <document2>, ... ],
  {
    writeConcern: <document>,
    ordered: <boolean>
  }
)
```

4. **Example:** Here is an example of how to create a document in MongoDB using the `insertOne()` method:

```
db.users.insertOne(
  {
    name: "John Doe",
    age: 25,
    email: "john.doe@example.com"
  }
)
```

This will insert a new document into the `users` collection with the specified field values.

5. **Conclusion:** Creating documents in MongoDB is a simple process that can be done using the `insertOne()` or `insertMany()` methods. These methods allow you to insert single or multiple documents into a collection at once. It is important to understand the syntax and usage of these methods in order to effectively create documents in MongoDB.

Updating Documents in MongoDB

MongoDB provides several methods to update documents in a collection. Here are some key points to remember when updating documents in MongoDB:

1. The `updateOne()` method updates a single document that matches the specified filter.
2. The `updateMany()` method updates all documents that match the specified filter.
3. The `$set` operator is used to update the value of a field in a document.
4. The `$inc` operator is used to increment the value of a field in a document.
5. The `$push` operator is used to add an element to an array field in a document.
6. The `$pull` operator is used to remove an element from an array field in a document.
7. The `upsert` option can be used to insert a new document if no document matches the specified filter.
8. The `multi` option can be used to update multiple documents that match the specified filter.

Deleting Documents in MongoDB

MongoDB provides several methods to delete documents from a collection:

1. **`deleteOne()`**: This method deletes a single document that matches the specified filter. If multiple documents match the filter, only the first document is deleted.
2. **`deleteMany()`**: This method deletes all documents that match the specified filter.
3. **`findOneAndDelete()`**: This method finds a single document that matches the specified filter and deletes it. It also returns the deleted document.

Here is an example of how to use the `deleteOne()` method to delete a document from a collection:

```
db.collectionName.deleteOne({ field: value });
```

In this example, `collectionName` is the name of the collection from which you want to delete the document, and `{ field: value }` is the filter that specifies the document to delete.

Similarly, you can use the `deleteMany()` method to delete multiple documents from a collection:

```
db.collectionName.deleteMany({ field: value });
```

And you can use the `findOneAndDelete()` method to find and delete a single document from a collection:

```
db.collectionName.findOneAndDelete({ field: value });
```

It is important to note that deleting documents from a collection is a permanent action and cannot be undone. Therefore, it is recommended to use caution when

deleting documents from a collection.

Querying Documents in MongoDB

MongoDB is a NoSQL database that stores data in the form of documents. Querying documents in MongoDB involves specifying a set of criteria that the documents must meet in order to be returned as a result of the query. Here are some key points to keep in mind when querying documents in MongoDB:

1. **Filtering:** You can filter the documents returned by a query by specifying conditions in the query. For example, you can use the `$eq` operator to find documents where a specific field is equal to a certain value.
2. **Projection:** You can use projection to specify which fields to include or exclude from the documents returned by a query. This can be useful when you only need certain fields from the documents.
3. **Sorting:** You can sort the documents returned by a query by specifying a sort order for one or more fields. For example, you can use the `sort()` method to sort the documents by a specific field in ascending or descending order.
4. **Limiting:** You can limit the number of documents returned by a query by using the `limit()` method. This can be useful when you only need a certain number of documents from the result set.
5. **Aggregation:** MongoDB provides a powerful aggregation framework that allows you to perform complex data manipulation and analysis on the documents in a collection. You can use aggregation to group, filter, and transform documents in a collection.

These are some of the key concepts to keep in mind when querying documents in MongoDB. By using these techniques, you can retrieve the data you need from your MongoDB collections in an efficient and flexible manner.

Indexing in MongoDB

- Indexing is a way to optimize the performance of a database by minimizing the number of disk accesses required when a query is executed.
- MongoDB supports several types of indexes, including single field, compound, multikey, geospatial, text, and hashed indexes.
- Indexes can be created on any field or combination of fields in a MongoDB collection.
- Indexes are created using the `createIndex()` method on a collection.
- Indexes can be created in the background, allowing the database to continue to serve queries while the index is being built.
- Indexes can be dropped using the `dropIndex()` method on a collection.
- MongoDB automatically creates an index on the `_id` field of every collection.

- Indexes can be used to enforce uniqueness on a field or combination of fields in a collection.
- Indexes can be used to improve the performance of queries that use the indexed fields in their filter or sort criteria.
- Indexes can be used to improve the performance of aggregation operations that use the indexed fields in their pipeline stages.
- Indexes can be monitored and managed using the `db.collection.getIndexes()` and `db.collection.stats()` methods.
- Indexes can have a significant impact on the performance of a MongoDB database, and it is important to carefully design and manage indexes to ensure optimal performance.

Aggregation in MongoDB

Aggregation in MongoDB is the process of grouping data from multiple documents and performing various operations on the grouped data to return a single result. It is similar to the GROUP BY clause in SQL.

Some key points to remember about aggregation in MongoDB are:

1. Aggregation operations can be performed on a collection using the `aggregate()` method.
2. The `aggregate()` method takes an array of aggregation pipeline stages as its argument.
3. Each stage in the pipeline performs a specific operation on the data, such as filtering, grouping, or projecting.
4. The output of one stage is passed as input to the next stage in the pipeline.
5. The final result of the aggregation is returned by the last stage in the pipeline.

Some common aggregation pipeline stages are:

- **\$match:** Filters the documents to pass only those that match the specified condition(s) to the next stage in the pipeline.
- **\$group:** Groups the input documents by a specified `_id` expression and applies an accumulator expression to each group to compute the desired result.
- **\$project:** Passes along the documents with only the specified fields to the next stage in the pipeline.
- **\$sort:** Sorts all input documents and returns them to the pipeline in the specified order.
- **\$limit:** Limits the number of documents passed to the next stage in the pipeline.

Aggregation in MongoDB is a powerful tool that allows you to perform complex data analysis and manipulation on your data. It is important to understand the various stages and operations available in the aggregation pipeline to effectively use this tool.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order.
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection.
- You must create capped collections explicitly using the `db.createCollection()` method, which is a `mongosh` helper for the `create` command.
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection.
- Capped collection in MongoDB is basically used to store log information, the high volume of data, and cache information.
- After creation, if we try to add more than documents to their capacity, it overwrites the existing documents.

Spark

- Apache Spark is an open-source distributed computing system.
- It is designed to process large volumes of data in parallel across a cluster of computers.
- Spark can process data in memory, making it faster than traditional disk-based systems like Hadoop.
- It supports a wide range of data processing tasks, including SQL queries, machine learning, and graph processing.
- Spark has a rich set of APIs for different languages, including Scala, Java, Python, and R.
- It has a thriving community of developers and users, with regular releases and updates.
- Spark can be deployed on a variety of platforms, including standalone, on-premises clusters, and cloud-based services.
- It is widely used in industry and academia for big data processing and analysis.

Installing Spark

1. **Download Spark:** You can download the latest version of Spark from the Apache Spark website. Choose the package type that is suitable for your system and download it.
2. **Install Spark:** After downloading the package, extract it to a location on your system. You can do this by using the `tar` command on Linux or macOS, or by using a program like 7-Zip on Windows.
3. **Set up environment variables:** You need to set up a few environment variables to use Spark. These include `SPARK_HOME`, which should be set to the location where you extracted the Spark package, and `PATH`, which should include the `bin` directory of the Spark installation.

4. **Test the installation:** To test if the installation was successful, open a terminal or command prompt and type `spark-shell`. This should start the Spark shell, where you can interactively run Spark commands.
5. **Configure Spark:** You can configure Spark by editing the `spark-defaults.conf` file, which is located in the `conf` directory of the Spark installation. This file contains default settings for Spark, which you can modify to suit your needs.
6. **Start using Spark:** After completing the above steps, you are ready to start using Spark. You can use the `spark-submit` command to run Spark applications, or use the `spark-shell` to interactively explore data using Spark.

Spark Applications

- Spark Applications consist of a driver process and a set of executor processes.
- The driver process runs your main () function, sits on a node in the cluster, and is responsible for three things: maintaining information about the Spark Application; responding to a user's program or input; and analyzing, distributing, and scheduling work across the executors.
- Spark applications run as independent sets of processes on a cluster, coordinated by the driver program.
- The driver consists of your program, like a C# console app, and a Spark session. The Spark session takes your program and divides it into smaller tasks that are handled by the executors.
- A Spark application runs as independent processes, coordinated by the SparkSession object in the driver program. The resource or cluster manager assigns tasks to workers, one task per partition. A task applies its unit of work to the dataset in its partition and outputs a new partition dataset.
- For an in-depth overview of the API, start with the RDD programming guide and the SQL programming guide, or see "Programming Guides" menu for other components. For running applications on a cluster, head to the deployment overview.
- Spark applications run as independent sets of processes on a pool, coordinated by the SparkContext object in your main program, called the driver program. The SparkContext can connect to the cluster manager, which allocates resources across applications. The cluster manager is Apache Hadoop YARN.

Jobs in Spark

- A job in Spark is a unit of work that is sent to the cluster for execution.
- Jobs are divided into stages, which are further divided into tasks.
- Each task is a unit of work that is sent to a single executor.

- Jobs are triggered by actions, such as `count()` or `collect()`.
- The number of stages in a job depends on the number of shuffle operations required.
- The Spark scheduler is responsible for scheduling jobs and managing their execution.
- Jobs can be monitored and managed using the Spark web UI or the Spark REST API.
- Jobs can be submitted to the cluster using the `spark-submit` command or the `SparkContext` API.
- Jobs can be cancelled using the `SparkContext.cancelJob()` or `SparkContext.cancelAllJobs()` methods.
- Jobs can be run in different modes, such as client mode or cluster mode, depending on the deployment configuration.

Stages and Tasks in Spark

- There are mainly two stages associated with the Spark frameworks such as, `ShuffleMapStage` and `ResultStage`.
- The Shuffle MapStage is the intermediate phase for the tasks which prepares data for subsequent stages, whereas `resultStage` is a final step to the spark function for the particular set of tasks in the spark job.
- A stage in Spark represents a set of parallel tasks that operate on a subset of the data.
- The tasks within a stage are executed on different executor nodes in the cluster, and the data is partitioned into smaller chunks called partitions, which are processed by the tasks in parallel.
- A Stage contains multiple tasks, and the tasks within a Stage can be executed in parallel on different nodes in the Spark cluster.
- The tasks within a Stage are dependent on each other and are executed in a specific order, so the output of one task is used as input for the next task.
- Each action operation will create one Spark job.
- Each Spark job will be converted to a DAG which includes one or more stages.
- A Spark stage is a smaller sets of tasks that depend on each other.
- Stages are created for each job based on shuffle boundaries, i.e. what operations can be performed.

Resilient Distributed Databases in Spark

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark. It is an immutable distributed collection of objects. Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster. RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .

RDDs are reliable and memory-efficient when it comes to parallel processing. By

storing and processing data in RDDs, Spark speeds up MapReduce processes .

At the core, an RDD is an immutable distributed collection of elements of your data, partitioned across nodes in your cluster that can be operated in parallel with a low-level API that offers transformations and actions .

Spark can create distributed datasets from any storage source supported by Hadoop, including your local file system, HDFS, Cassandra, HBase, Amazon S3, etc. Spark supports text files, SequenceFiles, and any other Hadoop Input-Format. Text file RDDs can be created using SparkContext's `textFile` method .

Anatomy of a Spark Job Run

1. **Client Mode:** In client mode, the driver program runs on the client machine, and the application master is only used for requesting resources from YARN.
2. **Cluster Mode:** In cluster mode, the driver program runs on a worker node inside the cluster, and the application master is used for both requesting resources from YARN and managing the driver program.
3. **Job Submission:** When a Spark job is submitted, the driver program converts the user's code into a series of tasks to be executed on the cluster.
4. **Task Scheduling:** The driver program communicates with the cluster manager to schedule tasks on the worker nodes.
5. **Task Execution:** Each task is executed on a worker node, and the results are sent back to the driver program.
6. **Shuffling:** If the tasks require data to be exchanged between worker nodes, a shuffle operation is performed.
7. **Result Collection:** Once all tasks have completed, the driver program collects the results and returns them to the user.

Spark on YARN

- Apache Spark is a fast and general-purpose cluster computing system.
- It provides high-level APIs in Java, Scala, Python and R, and an optimized engine that supports general execution graphs.
- YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop.
- Spark can run on YARN, which allows it to integrate with Hadoop environments and access Hadoop data.
- To run Spark on YARN, the following requirements must be met:
 - Spark must be built with YARN support.
 - The Hadoop configuration must be available on the Spark driver and executor machines.

- The Hadoop configuration must specify the YARN resource manager address and scheduler address.
- When running Spark on YARN, the Spark driver runs in the client machine or in a YARN application master container.
- The Spark executors run in YARN containers.
- Spark on YARN supports dynamic allocation of cluster resources, which allows it to scale up and down based on the workload.
- Spark on YARN also supports running in client mode or cluster mode.
- In client mode, the Spark driver runs on the client machine, while in cluster mode, the driver runs in a YARN application master container.
- Running Spark on YARN provides several benefits, including integration with Hadoop data and security, and the ability to dynamically allocate resources based on the workload.

SCALA

Scala is a general-purpose, high-level, multi-paradigm programming language. It is a pure object-oriented programming language which also provides support to the functional programming approach. Scala combines object-oriented and functional programming in one concise, high-level language. Scala's static types help avoid bugs in complex applications, and its JVM and JavaScript runtimes let you build high-performance systems with easy access to huge ecosystems of libraries. Scala programs can convert to bytecodes and can run on the JVM (Java Virtual Machine). Designed to be concise, many of Scala's design decisions are aimed to address criticisms of Java.

- Scala is a strong statically typed high-level general-purpose programming language.
- Scala supports both object-oriented programming and functional programming.
- Scala's design decisions are aimed to address criticisms of Java.
- Scala's static types help avoid bugs in complex applications.
- Scala's JVM and JavaScript runtimes let you build high-performance systems with easy access to huge ecosystems of libraries.
- Scala programs can convert to bytecodes and can run on the JVM (Java Virtual Machine).

Introduction to Scala

Scala is a modern, multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way. It smoothly integrates features of object-oriented and functional languages, enabling developers to be more productive while retaining full interoperability with Java.

Some key features of Scala include: - Scala is a statically typed language, which means that the type of a variable is checked at compile-time. - Scala supports

both object-oriented and functional programming paradigms. - Scala has a concise and expressive syntax, allowing developers to write less code while achieving the same functionality. - Scala is fully interoperable with Java, meaning that Scala code can use Java libraries and vice versa. - Scala has a sophisticated type system, supporting features such as generic classes, variance annotations, and higher-kinded types.

Scala is used by many companies, including Twitter, LinkedIn, and Netflix, and has a growing community of developers and users. It is a popular choice for building scalable, concurrent, and distributed systems.

Classes and Objects in Scala

- **Classes** in Scala are blueprints for creating objects. They can contain methods, values, variables, types, objects, traits, and classes which are collectively called members.
- A **class** is defined using the **class** keyword, followed by the class name and a parameter list.
- The body of the class is defined in curly braces `{}` and can contain class members.
- To create an **object** of a class, the **new** keyword is used, followed by the class name and any required parameters.
- **Objects** are instances of classes and can access the members of their class.
- Scala also has a concept of **singleton objects**, which are defined using the **object** keyword. Singleton objects are used to hold single instances of a class and cannot be instantiated using the **new** keyword.
- Singleton objects can have the same name as a class, in which case they are called **companion objects**. Companion objects and their corresponding classes can access each other's private members.
- Classes and objects in Scala can also make use of **inheritance**, allowing them to inherit members from a superclass. Inheritance is achieved using the **extends** keyword.
- Scala also supports **traits**, which are similar to interfaces in other languages. Traits can contain both abstract and concrete members and can be mixed into classes using the **with** keyword.

Basic Types and Operators in Scala

Scala has a rich set of built-in data types and operators. Here are some of the basic types and operators in Scala:

1. **Numeric Types:** Scala has several numeric types, including **Byte**, **Short**, **Int**, **Long**, **Float**, and **Double**. These types represent 8-bit, 16-bit, 32-bit, and 64-bit signed integers, as well as 32-bit and 64-bit floating-point numbers, respectively.
2. **Boolean Type:** The **Boolean** type in Scala has two values: **true** and **false**.

3. **Character Type:** The `Char` type in Scala represents a 16-bit Unicode character.
4. **String Type:** The `String` type in Scala represents a sequence of characters.
5. **Arithmetic Operators:** Scala supports the standard arithmetic operators, including `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (modulus).
6. **Relational Operators:** Scala also supports the standard relational operators, including `==` (equal to), `!=` (not equal to), `>` (greater than), `<` (less than), `>=` (greater than or equal to), and `<=` (less than or equal to).
7. **Logical Operators:** The logical operators in Scala include `&&` (logical AND), `||` (logical OR), and `!` (logical NOT).
8. **Bitwise Operators:** Scala supports bitwise operators, including `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), `~` (bitwise NOT), `<<` (left shift), `>>` (right shift), and `>>>` (unsigned right shift).

Built-in Control Structures in Scala

Scala has several built-in control structures that allow you to control the flow of your program. These include:

1. **If-else statements:** This is a conditional statement that allows you to execute a block of code if a certain condition is met, and another block of code if the condition is not met.
2. **While loops:** This is a loop that allows you to execute a block of code repeatedly while a certain condition is met.
3. **For loops:** This is a loop that allows you to iterate over a collection of elements, executing a block of code for each element in the collection.
4. **Match expressions:** This is a powerful control structure that allows you to match a value against a series of patterns, executing a block of code for the first pattern that matches the value.
5. **Try-catch-finally expressions:** This is a control structure that allows you to handle exceptions in your code, executing a block of code if an exception is thrown, and another block of code whether or not an exception is thrown.

These control structures are similar to those found in other programming languages, but Scala provides some additional features and syntactic sugar that make them more powerful and easier to use. For example, Scala's `for` loop can be used to iterate over multiple collections simultaneously, and its `match` expression can be used to destructure complex data structures. Additionally, Scala's

control structures are expressions, meaning that they return a value, which can be useful in certain situations.

Functions and Closures in Scala

- **Functions** in Scala are objects that can be assigned to variables, passed as arguments to other functions, and returned as values from other functions.
- Functions can be defined using the **def** keyword, followed by the function name, parameter list, return type, and function body.
- Functions can also be defined as **anonymous functions**, also known as **function literals**, which are functions without a name. They can be assigned to variables or passed as arguments to other functions.
- **Closures** are functions that can access variables from their enclosing scope, even if the function is invoked outside of that scope.
- Closures are useful for creating functions that can operate on data that is not passed as an argument to the function.
- In Scala, closures are created automatically when a function accesses a variable from its enclosing scope.
- The variables accessed by the closure are said to be **captured** by the closure, and the closure maintains a reference to those variables even if they go out of scope.
- Closures can be used to create functions with **state**, where the state is maintained in the captured variables.

Inheritance in Scala

Inheritance is a fundamental concept in object-oriented programming that allows the creation of hierarchical relationships between classes. In Scala, inheritance works similarly to other object-oriented languages such as Java.

Here are some key points to remember about inheritance in Scala:

1. A subclass can inherit from a superclass by using the **extends** keyword. The subclass will inherit all non-private members of the superclass.
2. Scala supports single inheritance, meaning that a class can only extend one superclass.
3. A subclass can override methods of the superclass by using the **override** keyword. The method in the subclass must have the same signature as the method in the superclass.
4. A subclass can call the constructor of the superclass by using the **super** keyword.
5. Scala also supports abstract classes, which are classes that cannot be instantiated and can contain abstract methods that must be implemented by subclasses.
6. Traits are another way to achieve inheritance in Scala. A trait is similar to an interface in Java and can contain both abstract and concrete methods. A class can extend multiple traits by using the **with** keyword.

These are some of the key points to remember about inheritance in Scala. It is a powerful feature that allows for code reuse and the creation of complex class hierarchies. It is important to use inheritance judiciously and follow good design principles to avoid creating overly complex and difficult to maintain code.

Hadoop Eco System Frameworks, Pig, Hive, and HBase

- **Hadoop** includes several additional modules that provide additional functionality, such as **Hive**, **Pig**, and **HBase**.
- **HBase** is a scalable, distributed database that supports structured data storage for large tables.
- **Hive** is a data warehouse infrastructure that provides data summarization and ad-hoc querying. Its query language is called HQL (Hive Query Language) .
- **Pig** is a high-level data-flow language and execution framework for parallel computation. It helps to achieve ease of programming and optimization .
- **HBase** is a distributed column-oriented database built on top of the HDFS. It is an open-source project and horizontally scalable. HBase is a data model that is similar to Google's big table that designed to provide quick random access to huge amounts of structured data.

Hadoop Eco System Frameworks

Hadoop is a framework that enables processing of large data sets which reside in the form of clusters. Being a framework, Hadoop is made up of several modules that are supported by a large ecosystem of technologies.

- **Introduction:** Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions.
- **Major Elements:** There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common.
- **Hadoop Distributed File System (HDFS):** The core component of the Hadoop ecosystem is a Hadoop distributed file system (HDFS). HDFS is a distributed file system that has the capability to store a large stack of data sets.
- **Parallelism:** Hadoop provides massive parallelism with low latency and high throughput, which makes it well-suited for big data problems.

Applications on Big Data using Pig

1. **Data Processing:** Pig is used for processing large datasets. It can handle structured, semi-structured, and unstructured data. Pig scripts can be used to perform ETL (Extract, Transform, Load) tasks, data cleansing, and data preparation for analysis.
2. **Data Analysis:** Pig can be used for data analysis tasks such as data

aggregation, data filtering, data grouping, and data sorting. It can also be used for data mining and machine learning tasks.

3. **Data Storage:** Pig can be used to store data in various formats such as CSV, JSON, and XML. It can also be used to store data in Hadoop Distributed File System (HDFS) and other distributed storage systems.
4. **Data Integration:** Pig can be used to integrate data from multiple sources. It can be used to join data from different sources, merge data, and perform data enrichment.
5. **Data Visualization:** Pig can be used to generate data visualizations such as charts, graphs, and maps. It can be used to create interactive dashboards and reports.

Applications on Big Data using Hive

Hive is a data warehouse software project that facilitates reading, writing, and managing large datasets residing in distributed storage using SQL. It is built on top of Apache Hadoop, which is an open-source framework used for distributed storage and processing of big data. Here are some applications of Big Data using Hive:

1. **Data Mining:** Hive can be used for data mining purposes to discover patterns and trends in large datasets. It can be used to run SQL-like queries on structured and semi-structured data, making it easier to extract valuable insights from big data.
2. **Data Summarization:** Hive can be used to summarize large datasets and generate reports. It can be used to aggregate data and calculate metrics such as averages, sums, and counts, which can be used to generate reports and visualizations.
3. **Data Analysis:** Hive can be used for data analysis purposes to analyze large datasets and generate insights. It can be used to run complex analytical queries on big data, making it easier to extract valuable insights and make data-driven decisions.
4. **ETL Processing:** Hive can be used for ETL (Extract, Transform, Load) processing to move and transform data between different systems. It can be used to extract data from various sources, transform it into a desired format, and load it into a target system for further analysis.
5. **Machine Learning:** Hive can be used in conjunction with machine learning algorithms to build predictive models on big data. It can be used to preprocess and transform data, making it easier to train machine learning models on large datasets.

Applications on Big Data using HBase

HBase is an open-source, non-relational, distributed database that is modeled after Google's Bigtable and is written in Java. It is developed as part of the Apache Software Foundation's Apache Hadoop project and runs on top of HDFS (Hadoop Distributed File System). HBase is used for real-time read/write access to large datasets.

Some of the applications of HBase in Big Data include:

1. **Real-time data analytics:** HBase can be used to store and analyze large volumes of data in real-time. This is useful for applications such as fraud detection, where quick response times are critical.
2. **Data warehousing:** HBase can be used as a data store for large-scale data warehousing applications. It can store and retrieve large volumes of structured and unstructured data, making it a good choice for applications that require fast access to large datasets.
3. **Log storage and analysis:** HBase can be used to store and analyze log data from various sources. This is useful for applications such as web analytics, where large volumes of log data need to be processed and analyzed in real-time.
4. **Recommendation engines:** HBase can be used to store and retrieve data for recommendation engines. This is useful for applications such as e-commerce websites, where personalized recommendations can be generated based on user behavior.
5. **Social media analytics:** HBase can be used to store and analyze data from social media platforms. This is useful for applications such as sentiment analysis, where large volumes of data need to be processed and analyzed in real-time.

Overall, HBase is a powerful tool for managing and analyzing large volumes of data in real-time. Its ability to handle structured and unstructured data, combined with its scalability and performance, make it a popular choice for Big Data applications.

Pig

- The pig (*Sus domesticus*), often called swine, hog, or domestic pig when distinguishing from other members of the genus *Sus*, is an omnivorous, domesticated, even-toed, hoofed mammal. It is variously considered a subspecies of *Sus scrofa* (the wild boar or Eurasian boar) or a distinct species .
- Pigs are stout-bodied, short-legged, omnivorous mammals, with thick skin usually sparsely coated with short bristles. Their hooves have two functional and two nonfunctional digits. Domestic North American pigs originated from wild stocks still found in European, Asian, and North African forests .

- Any of various mammals of the family Suidae, having short legs, hooves with two weight-bearing toes, bristly hair, and a cartilaginous snout used for digging, including the domesticated hog (*Sus scrofa* subsp. *domestica* syn. *S. domesticus*) and wild species such as the bushpig .

Pig - Introduction to PIG

1. Pig is a high-level platform for creating MapReduce programs used with Hadoop.
2. The language for this platform is called Pig Latin.
3. Pig Latin abstracts the programming from the Java MapReduce idiom into a notation which makes MapReduce programming high level, similar to that of SQL for RDBMS systems.
4. Pig Latin can be extended using UDF (User Defined Functions).
5. Pig is generally used with Hadoop; you can perform all the data manipulation operations in Hadoop using Pig.
6. Pig is an open-source project under the Apache Software Foundation.
7. Pig is a data flow language.
8. Pig is designed to handle any kind of data, structured or unstructured.
9. Pig is a procedural language, where the programmer has to define the steps to achieve the desired result.
10. Pig is a platform to analyze large data sets that consist of a high-level language for expressing data analysis programs, coupled with infrastructure for evaluating these programs.

Execution Modes of Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It has two execution modes:

1. **Local Mode:** In this mode, Pig runs on a single machine without requiring Hadoop or HDFS. It is used for development and testing of Pig scripts.
2. **MapReduce Mode:** In this mode, Pig runs on a Hadoop cluster and requires Hadoop and HDFS. It is used for processing large data sets.

In both modes, Pig scripts are translated into a series of MapReduce jobs that are run on the Hadoop cluster. The choice of execution mode depends on the size of the data set and the resources available for processing. For small data sets, local mode is sufficient, while for large data sets, MapReduce mode is more appropriate.

Comparison of Pig with Databases

- Pig is a high-level platform for creating MapReduce programs used with Hadoop, while databases are software systems that store, organize, and retrieve data.

- Pig Latin is a procedural language, whereas SQL used in databases is a declarative language.
- Pig is designed to handle large datasets, while databases can handle both small and large datasets.
- Pig can process both structured and unstructured data, while databases are designed to handle structured data.
- Pig is used for data analysis, while databases are used for data storage and retrieval.
- Pig can handle data in various formats, while databases require data to be in a specific format.
- Pig is used for batch processing, while databases can handle both batch and real-time processing.
- Pig is used for data processing in a distributed environment, while databases can be used in both distributed and non-distributed environments.

Grunt in Pig

- Grunt shell is a shell command.
- The Grunts shell of Apache Pig is mainly used to write Pig Latin scripts.
- Pig script can be executed with grunt shell which is native shell provided by Apache Pig to execute pig queries.
- We can invoke any shell commands using sh and fs.
- Using sh command from the Grunt shell, we cannot execute the commands that are a part of the shell environment (ex – cd).
- A study into pig behavior has found that grunts reflect their environment, and reveal their personalities.
- Proactive pigs grunt faster than reactive pigs.
- Male pigs grunt less if their environment is not enriching.
- The grunts made by pigs vary depending on the pig's personality and can convey important information about the welfare of this highly social species.
- A database of thousands of grunts, oinks, snuffles and squeals, gathered and recorded from hundreds of pigs at every stage of their lives, will be used to help farmers better understand their animals' emotional states and improve their welfare in the future.

Pig Latin

Pig Latin is a language game in which words in English are altered. The objective is to conceal the words from others not familiar with the rules. The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.

Here are the rules for translating English words into Pig Latin: 1. For words that begin with consonant sounds, all letters before the initial vowel are placed at the end of the word sequence. Then, “ay” is added, as in the following

examples: * “pig” → “igpay” * “latin” → “atinlay” * “banana” → “ananabay” * “happy” → “appyhay” * “duck” → “uckday” 2. For words that begin with vowel sounds, one just adds “way” or “yay” to the end (or just “ay”). Examples are: * “eat” → “eatway” or “eatay” * “omelet” → “omeletway” or “omeletay” * “are” → “areway” or “areay” * “egg” → “eggway” or “eggay” * “explain” → “explainway” * “always” → “alwaysway” or “alwaysay” * “ends” → “endsway” or “endsay” * “honest” → “honestway” * “I” → “Iway”

User Defined Functions in Pig

- User Defined Functions (UDFs) in Pig allow users to write their own functions to perform operations on data that are not supported by built-in Pig functions.
- UDFs can be written in several programming languages, including Java, Python, and Ruby.
- To use a UDF in a Pig script, the user must first register the JAR file containing the UDF with the **REGISTER** command.
- Once registered, the UDF can be invoked in the script using its fully qualified class name.
- UDFs can be used in various parts of a Pig script, including **FOREACH** statements, **FILTER** statements, and **GROUP BY** statements.
- UDFs can take one or more input parameters and can return a single value or a tuple.
- UDFs can be used to perform complex data transformations, data cleansing, and data enrichment tasks.
- UDFs provide a powerful and flexible way for users to extend the functionality of Pig and perform custom data processing tasks.

Data Processing Operators in Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It includes a language called Pig Latin for expressing data analysis programs. Pig Latin includes several data processing operators that can be used to perform various data manipulation tasks. Some of the most commonly used data processing operators in Pig are:

1. **LOAD**: This operator is used to load data from the file system into a Pig relation.
2. **STORE**: This operator is used to store the data of a Pig relation into the file system.
3. **FILTER**: This operator is used to filter out unwanted rows from a relation based on a specified condition.
4. **FOREACH**: This operator is used to generate a new relation by applying a transformation to each row of an input relation.
5. **GROUP**: This operator is used to group the rows of a relation based on one or more columns.

6. **JOIN**: This operator is used to join two or more relations based on a common column.
7. **ORDER**: This operator is used to sort the rows of a relation based on one or more columns.
8. **DISTINCT**: This operator is used to remove duplicate rows from a relation.
9. **LIMIT**: This operator is used to limit the number of rows in a relation to a specified number.

These are some of the most commonly used data processing operators in Fig. They can be used in various combinations to perform complex data analysis tasks.

Hive

- Hive is a data warehousing and SQL-like query language for data stored in Hadoop files.
- Hive enables data summarization, querying, and analysis of data.
- Hive queries are written in HiveQL, which is a query language similar to SQL.
- Hive allows you to project structure on largely unstructured data.
- Hive Metastore (HMS) provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions.
- Hive is built on top of Apache Hadoop and supports storage on S3, ADLS, GS, etc. through HDFS.
- Hive is a critical component of many data lake architectures.

Apache Hive Architecture

Apache Hive is a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale. It is an open-source data warehousing tool for performing distributed processing and data analysis. It was developed by Facebook to reduce the work of writing the Java MapReduce program. Apache Hive uses a Hive Query language, which is a declarative language similar to SQL .

The major components of Apache Hive are: - **Hive clients**: The user interface for users to submit queries and other operations to the system . - **Hive services**: The Hive Server 2 accepts incoming requests from users and applications and creates an execution plan and auto generates a YARN job to process SQL queries . - **Processing framework and Resource Management**: Apache Hive uses the Hadoop processing framework and resource management for processing and analyzing data . - **Distributed Storage**: Apache Hive uses the Hadoop Distributed File System (HDFS) for storing and managing data . - **Hive Metastore (HMS)**: Provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions, and therefore it is a critical component of many data lake architectures .

In summary, Apache Hive is an open-source data warehousing tool that enables

analytics at a massive scale by leveraging the Hadoop ecosystem. It uses a declarative query language, similar to SQL, to process and analyze data stored in HDFS. Its architecture consists of several components, including Hive clients, Hive services, a processing framework and resource management, distributed storage, and a Hive Metastore.

Installing Hive

1. Hive is a data warehousing and SQL-like query language for Hadoop.
2. To install Hive, you must first have Hadoop installed and configured on your system.
3. Download the latest stable release of Hive from the Apache Hive website.
4. Unpack the downloaded tarball and move the extracted directory to a location of your choice.
5. Set the environment variable `HIVE_HOME` to the location of the Hive installation.
6. Add the Hive `bin` directory to your `PATH` environment variable.
7. Configure Hive by editing the `hive-site.xml` file located in the `conf` directory of the Hive installation.
8. Start the Hive shell by running the `hive` command.
9. Verify that Hive is installed and working correctly by running some basic HiveQL commands.

Hive Shell

- Hive Shell provides Hive on OS remote access to your workers using the Hive on OS network infrastructure .
- It offers unique features, such as access via an SSH client and console sharing .
- Hive Shell is a shell utility that can be used to run Hive queries in either interactive or batch mode .
- HiveServer2 (introduced in Hive 0.11) has its own CLI called Beeline, which is a JDBC client based on SQLLine .
- Variables can be set in Hive scripts using the `set hivevar:tablename=mytable;` command .
- The `source` command can be used to bring a script into Hive, for example: `hive> source /path/to/setup.hql;` .
- Variables can be used in queries, for example: `hive> select * from $ {tablename}` or `hive> select * from $ {hivevar:tablename}` .

Hive Services

Hive provides several services to its users, including:

1. **HiveServer2:** This service provides a JDBC/ODBC server for clients to connect to and execute queries.

2. **Hive Metastore:** This service stores metadata about the tables, partitions, columns, and storage information for the data stored in Hive.
3. **Hive Web Interface:** This service provides a web-based user interface for users to interact with Hive and execute queries.
4. **Hive CLI:** This service provides a command-line interface for users to interact with Hive and execute queries.
5. **Beeline:** This service provides a JDBC client for users to connect to HiveServer2 and execute queries.

These services work together to provide a comprehensive and user-friendly interface for interacting with and querying data stored in Hive.

Hive Metastore

- Hive Metastore (HMS) is a service that stores metadata related to Apache Hive and other services, in a backend RDBMS, such as MySQL or PostgreSQL .
- Impala, Spark, Hive, and other services share the metastore .
- The connections to and from HMS include HiveServer, Ranger, and the NameNode that represents HDFS .
- HMS is a central repository of metadata for Hive tables and partitions in a relational database, and provides clients (including Hive, Impala and Spark) access to this information using the metastore service API.
- HMS provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions, and therefore it is a critical component of many data lake architectures.
- Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs etc though hdfs.
- Hive Metastore was developed as a part of Apache Hive, “a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale”.
- Hive achieves this goal by being the storage point for all the meta-information about your data storages.

Comparison of Hive with traditional databases

Hive is a data warehouse software system that provides data query and analysis. It gives an interface like SQL to query data stored in various databases and file systems that integrate with Hadoop. Hive helps with querying and managing large datasets real fast .

- **Schema on Read:** Hive enforces schema on read time, meaning it does not verify the schema until the data is read .
- **Scalability:** Hive is very easily scalable at low cost .
- **Based on Hadoop:** Hive is based on Hadoop notation that is Write once Read many times .
- **Record level updates:** Record level updates are not possible in Hive .

- **Data Analysis:** Hive is majorly used to do analysis on a huge amount of data which traditional databases cannot process using MapReduce .

On the other hand, traditional databases enforce schema on write time, meaning the table schema is enforced when the data is written . They are not as easily scalable and can be costly to scale up . Record level updates, insertions, and deletions are possible in traditional databases .

In summary, Hive is similar to traditional databases by supporting the SQL interface but it is not a full database. Instead of calling Hive a database, it can also be called a data warehouse . It is better suited for querying and managing large datasets, while traditional databases are better suited for record level updates and transactions.

HiveQL

- Hive Query Language (HiveQL) is a query language in Apache Hive for processing and analyzing structured data .
- It separates users from the complexity of Map Reduce programming .
- It reuses common concepts from relational databases, such as tables, rows, columns, and schema, to ease learning .
- Hive provides Built-in operators for Data operations to be implemented on the tables present inside Hive warehouse .
- HiveQL provides the basic SQL like operations. SELECT statement is used to retrieve the data from a table .
- Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs etc though hdfs .
- Hive provides the necessary SQL abstraction to integrate SQL-like queries (HiveQL) into the underlying Java without the need to implement queries in the low-level Java API .
- Since most data warehousing applications work with SQL-based querying languages, Hive aids portability of SQL-based applications to Hadoop .
- Apache Hive is a data warehouse system for Apache Hadoop. Hive enables data summarization, querying, and analysis of data .
- Hive queries are written in HiveQL, which is a query language similar to SQL .
- Hive allows you to project structure on largely unstructured data .

Tables in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to create and manage tables in a relational database-like manner. Here are some key points to remember about tables in Hive:

1. Tables in Hive are similar to tables in a relational database, and can be created, altered, and dropped using HiveQL commands.
2. Hive supports two types of tables: managed tables and external tables. Managed tables are stored in the Hive warehouse directory, while external

tables are stored outside the warehouse directory and are used to access data stored in external sources.

3. Hive tables can be partitioned and bucketed to improve query performance.
4. Hive supports a variety of data formats, including text files, sequence files, and ORC files.
5. Hive tables can be accessed using HiveQL, as well as through external tools such as Apache Spark and Apache Pig.

Querying Data in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to query and analyze large datasets stored in Hadoop files. Here are some key points to remember when querying data in Hive:

1. Hive supports a SQL-like query language called HiveQL. It is similar to standard SQL, but with some differences.
2. HiveQL supports a wide range of data types, including primitive types (such as INT, STRING, and BOOLEAN), complex types (such as ARRAY, MAP, and STRUCT), and user-defined types.
3. HiveQL supports a wide range of functions, including aggregate functions (such as COUNT, SUM, and AVG), string functions (such as CONCAT and SUBSTR), and date/time functions (such as CURRENT_DATE and DATE_ADD).
4. HiveQL supports a wide range of operators, including arithmetic operators (such as + and -), comparison operators (such as = and >), and logical operators (such as AND and OR).
5. HiveQL supports a wide range of clauses, including SELECT, FROM, WHERE, GROUP BY, HAVING, and ORDER BY.
6. HiveQL supports subqueries, both in the WHERE clause and in the FROM clause.
7. HiveQL supports joins, including inner joins, left outer joins, right outer joins, and full outer joins.
8. HiveQL supports views, which are virtual tables based on the result of a SELECT statement.
9. HiveQL supports partitioning, which allows users to split large tables into smaller, more manageable partitions based on the values of one or more columns.
10. HiveQL supports bucketing, which allows users to split large tables into smaller, more manageable buckets based on the hash value of one or more columns.

These are some of the key points to remember when querying data in Hive. It is important to note that HiveQL is a powerful and flexible query language, and there are many more advanced features and capabilities that are beyond the scope of this brief overview. It is recommended to consult the Hive documentation for more detailed information.

User Defined Functions in Hive

- User Defined Functions (UDFs) are custom functions that can be created by users to perform specific operations on data in Hive.
- UDFs can be written in Java, Python, or other languages and can be used in Hive queries.
- UDFs can be used to perform complex data manipulation tasks that are not possible using built-in Hive functions.
- UDFs can be created using the **CREATE FUNCTION** statement in Hive.
- UDFs can be either temporary or permanent. Temporary UDFs are only available for the duration of the current session, while permanent UDFs are stored in the Hive metastore and can be used across sessions.
- UDFs can be used in Hive queries in the same way as built-in functions, by specifying the function name and passing the required arguments.
- UDFs can be used to extend the functionality of Hive and provide users with more flexibility in data manipulation.

Sorting and Aggregating in Hive

- **Sorting** in Hive refers to the process of arranging data in a specific order, either ascending or descending, based on one or more columns.
- Hive supports two types of sorting: **Order By** and **Sort By**.
- **Order By** sorts the data globally and returns the sorted data to the client. It is a costly operation as it requires shuffling of data between the nodes.
- **Sort By** sorts the data locally within each reducer. The final output may not be globally sorted, but the data within each reducer is sorted.
- **Aggregation** in Hive refers to the process of grouping data based on one or more columns and calculating aggregate functions such as sum, count, average, min, max, etc. on the grouped data.
- Hive supports several aggregate functions, including **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, etc.
- Aggregation can be performed using the **GROUP BY** clause, which groups the data based on the specified columns and calculates the aggregate functions on the grouped data.
- Hive also supports **HAVING** clause, which can be used to filter the grouped data based on the result of the aggregate functions.

Map Reduce scripts in Hive

- Hive is a data warehousing and SQL-like query language that can be used to process structured data in Hadoop.
- Hive can use MapReduce scripts to execute queries, which allows for the processing of large datasets in parallel.
- MapReduce is a programming model for processing large datasets in parallel across a cluster of computers.
- In Hive, MapReduce scripts can be written in various languages, including Java, Python, and Ruby.

- To use a MapReduce script in Hive, the script must be added to the Hive script file and the appropriate input and output formats must be specified.
- Hive can automatically generate MapReduce scripts for certain types of queries, which can simplify the process of writing and executing MapReduce scripts.
- MapReduce scripts in Hive can be used for a variety of tasks, including data transformation, filtering, and aggregation.
- Using MapReduce scripts in Hive can improve the performance of queries by distributing the processing across multiple nodes in the cluster.

Joins and Subqueries in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to write queries in a familiar SQL syntax, making it easier to work with large datasets stored in Hadoop. Hive supports several types of joins, including inner join, left outer join, right outer join, and full outer join.

- **Inner Join:** An inner join returns only the rows from both tables that satisfy the join condition. In Hive, the syntax for an inner join is as follows:

```
SELECT ...
FROM table1
JOIN table2
ON table1.column = table2.column
```

- **Left Outer Join:** A left outer join returns all the rows from the left table and the matching rows from the right table. If there is no match, the result will contain NULL values for all columns of the right table. The syntax for a left outer join in Hive is as follows:

```
SELECT ...
FROM table1
LEFT OUTER JOIN table2
ON table1.column = table2.column
```

- **Right Outer Join:** A right outer join returns all the rows from the right table and the matching rows from the left table. If there is no match, the result will contain NULL values for all columns of the left table. The syntax for a right outer join in Hive is as follows:

```
SELECT ...
FROM table1
RIGHT OUTER JOIN table2
ON table1.column = table2.column
```

- **Full Outer Join:** A full outer join returns all the rows from both tables. If there is no match, the result will contain NULL values for all columns of the table without a match. The syntax for a full outer join in Hive is as follows:

```
SELECT ...
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column
```

Subqueries are a powerful feature in SQL that allow you to nest one query inside another. Hive supports subqueries in the WHERE and HAVING clauses, as well as in the FROM clause as an inline view. Here is an example of a subquery in the WHERE clause:

```
SELECT ...
FROM table1
WHERE column1 IN (SELECT column2 FROM table2 WHERE ...)
```

In this example, the subquery returns a set of values that are used to filter the rows returned by the outer query. Subqueries can be used to perform complex filtering operations and to join data from multiple tables in a flexible manner.

HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS).
- It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.
- It is ideal for high-scale real-time applications, such as a social media app or a streaming application.
- Thanks to the lack of a fixed database schema in a non-relational database like HBase, developers can add new data without conforming to a schema model.
- HBase is an open-source, distributed, versioned, non-relational database modeled after Google's Bigtable.
- It provides Bigtable-like capabilities on top of Hadoop and HDFS.
- HBase is developed as part of Apache Software Foundation's Apache Hadoop project and can run on top of HDFS (Hadoop Distributed File System) or Alluxio.

HBase Concepts

HBase is a distributed, column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). It is an open-source, non-relational database that provides real-time read/write access to large datasets. Some key concepts of HBase include:

1. **Table:** An HBase table is a collection of rows, where each row is identified by a unique row key.

2. **Column Family:** A column family is a logical grouping of columns in an HBase table. All columns within a column family are stored together on disk.
3. **Column:** A column in HBase is identified by its column family and a column qualifier. The column qualifier is a string that distinguishes the column from other columns in the same column family.
4. **Cell:** A cell in HBase is the intersection of a row and a column. It contains a value and a timestamp.
5. **Region:** An HBase table is partitioned into regions, where each region contains a contiguous range of rows. Regions are the unit of distribution and load balancing in HBase.
6. **RegionServer:** A RegionServer is a process that manages a set of regions. It is responsible for handling read and write requests for the regions it manages.
7. **HMaster:** The HMaster is the master process in an HBase cluster. It is responsible for coordinating the activities of the RegionServers and for managing the metadata of the HBase cluster.

HBase Clients

HBase is a distributed, scalable, big data store that runs on top of the Hadoop Distributed File System (HDFS). It is modeled after Google's Bigtable and provides a way to store and access large amounts of structured and semi-structured data.

There are several ways to interact with HBase, including the following clients:

1. **HBase Shell:** This is a command-line interface that allows users to interact with HBase using commands. It is useful for administrative tasks and simple data manipulation.
2. **Java API:** HBase provides a Java API for programmatic access to the data stored in HBase. This API allows developers to create, read, update, and delete data in HBase from their Java applications.
3. **Thrift and REST APIs:** HBase also provides Thrift and REST APIs for non-Java clients. These APIs allow developers to interact with HBase using other programming languages such as Python, Ruby, and PHP.
4. **Apache Phoenix:** Apache Phoenix is a SQL skin for HBase. It allows users to interact with HBase using SQL commands, making it easier for users familiar with SQL to work with HBase.

Each of these clients has its own strengths and weaknesses, and the choice of client will depend on the specific needs of the user and the application. It is important to carefully evaluate the available options and choose the client that best meets the requirements of the project.

HBase Example

HBase is a distributed, column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). It is an open-source project that is part of the Apache Hadoop ecosystem. HBase is designed to provide random, real-time read/write access to large datasets.

Here is an example of how to create a table in HBase:

1. Start the HBase shell by running the command `hbase shell`.
2. In the HBase shell, use the `create` command to create a new table. For example, to create a table named `mytable` with a column family named `mycf`, you would enter the following command: `create 'mytable', 'mycf'`.
3. You can verify that the table was created by using the `list` command, which will show you a list of all the tables in HBase.

This is just a simple example of how to create a table in HBase. There are many other operations that you can perform, such as inserting data, querying data, and deleting data. You can learn more about these operations by consulting the HBase documentation.

HBase vs RDBMS

- **RDBMS** (Relational Database Management System) and **HBase** are both types of database management systems, but they differ in several ways.
- **Data Model:** RDBMS uses a relational data model, where data is stored in tables with predefined columns and rows. HBase, on the other hand, uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model .
- **Scaling:** RDBMS and HBase differ in their scaling capabilities. RDBMS is more suitable for traditional, transactional applications that require strong consistency, whereas HBase is better suited for big data applications that require horizontal scaling and high-speed processing .
- **Consistency:** In HBase, there is no transaction guaranty, whereas RDBMS mostly guarantees transaction integrity .
- **JOINS:** HBase supports JOINS, whereas RDBMS does not support JOINS .
- **Referential integrity:** HBase does not guarantee referential integrity, whereas RDBMS does .
- **Speed:** HBase is built on Hadoop and modeled after Google BigTable. HBase provides random access and strong consistency for large amounts of unstructured and semi-structured data in a schemaless database organized by column families .

Advanced Usage of HBase

HBase is a distributed, scalable, big data store that is used for random, real-time read/write access to large datasets. It is used in various fields and industries,

including healthcare, e-commerce, and sports, among others.

- In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area .
- In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights .
- In sports, HBase is used to store match details and the history of each match .

HBase has two fundamental key structures: the row key and the column key. Both can be used to convey meaning, by either the data they store, or by exploiting their sorting order. These keys can be used to solve commonly found problems when designing storage solutions .

It is important to note that HBase is not a traditional database system and does not have features such as typed columns, transactions, advanced query languages, or secondary indexes. It is best used when there is a need to store huge volumes of data and high scalability is required .

Schema Design in HBase

- HBase schema design is very different compared to the relation database schema design .
- HBase does not support any kind of joins, but it provides the single-indexing strategy on the row key .
- Each table in HBase table is indexed on row key. Data is sorted lexicographically by this row key .
- HBase schema design supports denormalization with nested entities .
- These nested entities are nothing but a column whose name is the unique identifier for the nested entity and whose value is the entire record mashed together .
- Since HBase allows dynamic column definition, there's no problem .

Advanced Indexing in HBase

- HBase is a column-oriented NoSQL database management system that runs on top of the Hadoop Distributed File System (HDFS). It is modeled after Google's Big Table and written in Java.
- In HBase, there are no indexes. The rowkey, column family, column qualifier are all stored in sort order based on the java comparable method for byte arrays.
- Access to records in any way other than through the primary row requires scanning over potentially all the rows in the table to test them against your filter.
- Secondary indexing is a way to improve the performance of HBase by creating an additional table to act as the index and update manually.

- HBase is a community-driven project, using Jira to help manage enhancements and bug fixes. HBASE-9203 is a Jira entry that exists specifically to address the ideas behind secondary Indexing.
- Lily HBase Indexer service embeds the NG-Data Indexer to provide a Near-Real-Time (NRT) resilient automated configuration-driven mechanism to trigger Morphline (Kite SDK) parsers over an HBase table.

Zookeeper

- A zookeeper is a professional who is responsible for the care and management of animals in a zoo.
- Zookeepers are responsible for the daily care of the animals, including feeding, cleaning, and providing enrichment.
- They also monitor the health and behavior of the animals and work with veterinarians to provide medical care when needed.
- Zookeepers may also be involved in breeding programs, research, and conservation efforts.
- To become a zookeeper, one typically needs a degree in a related field such as biology or animal science, as well as experience working with animals.
- The job can be physically demanding and may involve working outdoors in all weather conditions.
- Zookeepers must also have strong communication skills, as they often interact with the public and provide educational programs.
- The role of a zookeeper is crucial in ensuring the well-being of the animals in their care and in promoting conservation and education.

Zookeeper concepts

Apache ZooKeeper is a distributed coordination service that enables synchronization across a cluster. It is used to maintain configuration information, naming, and provide distributed synchronization and group services. Some of the key concepts of ZooKeeper are:

1. **Znodes:** ZooKeeper stores data in a hierarchical namespace, much like a file system. The data is stored in znodes, which are similar to files and directories. Each znode can store up to 1 MB of data and can have children.
2. **Watches:** Clients can set watches on znodes. When a znode changes, the client is notified of the change. This allows clients to be notified of changes in the data they are interested in.
3. **Ephemeral nodes:** Ephemeral nodes are znodes that are automatically deleted when the session that created them ends. This is useful for implementing locks and leader election.
4. **Sequential nodes:** Sequential nodes are znodes that are automatically assigned a unique monotonically increasing number by the ZooKeeper cluster.

ter. This is useful for implementing queues and other data structures that require unique identifiers.

5. **Consistency:** ZooKeeper guarantees that all clients will see the same view of the data at any given time. This is achieved through the use of a consensus protocol.
6. **Atomicity:** All updates to the ZooKeeper data are atomic. This means that either all the changes are applied, or none of them are.
7. **Reliability:** ZooKeeper is designed to be highly reliable. It can survive the failure of any single node in the cluster. If a majority of the nodes are available, the service will continue to operate.

How Zookeeper helps in monitoring a cluster

- Zookeeper helps maintain configuration information, naming, and group services for distributed applications .
- It implements different protocols on the cluster so that the application should not implement on their own .
- Zookeeper provides a single coherent view of multiple machines .
- As long as a majority of hosts are up, the service will be available .
- Zookeeper monitoring helps make sure the total node count inside the ZooKeeper tree is consistent .
- Zookeeper is one of the best cluster coordination systems that make use of the most durable synchronization methods to maintain the cluster's nodes' flawless connectivity .
- Running a Prometheus monitoring service is the easiest way to ingest and record ZooKeeper's metrics .
- There are several top ZooKeeper monitoring tools available, including Sematext, Prometheus & Grafana, ManageEngine, Site24x7, OpsView, Instana, SignalFx, Datadog, and AppDynamics .

How to build applications with Zookeeper

1. **Starting ZooKeeper and Application Builder:** After installing Application Builder in admin mode, on the server(s) where ZooKeeper is installed, execute the `/etc/init.d/zookeeper-service-default` command . If you have installed Application Builder in non-admin mode, start the ZooKeeper server before starting Application Builder. To start the ZooKeeper server on a Linux system, use the `Zookeeper/zookeeper/bin/zkServer.sh restart` command from your Watson Explorer installation directory .
2. **Running ZooKeeper on Kubernetes:** You can use the `kubectl apply` command to create the manifest. For example, `kubectl apply -f https://k8s.io/examples/application/zookeeper/zookeeper.yaml`.

This creates the **zk-hs** Headless Service, the **zk-cs** Service, the **zk-pdb** PodDisruptionBudget, and the **zk** StatefulSet .

3. **Using ZooKeeper in Distributed Applications:** ZooKeeper is used by Apache HBase, HDFS, and other Apache Hadoop projects to provide highly-available services and to make distributed programming easier. You can use ZooKeeper to easily and safely implement important features in your distributed software .
4. **Setting up a ZooKeeper server in standalone mode:** The server is contained in a single JAR file, so installation consists of creating a configuration. Once you've downloaded a stable ZooKeeper release, unpack it and `cd` to the root. To start ZooKeeper, you need a configuration file .
5. **Getting a ZooKeeper distribution:** Download a recent stable release from one of the Apache Download Mirrors. Setting up a ZooKeeper server in standalone mode is straightforward. The server is contained in a single JAR file, so installation consists of creating a configuration .

IBM Big Data strategy

- IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy, where the company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .
- Smarter Planet was a corporate initiative of IBM, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM suggests a six-step process to design a data strategy: 1) Understand your business objectives, 2) Assess your current state, 3) Map out data strategy framework, 4) Define your data's target, 5) Implement your data strategy, and 6) Monitor and adjust your data strategy.
- IBM also recommends giving data assets and accelerators top priority, developing a process and culture around data that enables true standardization, re-use, portability, speed to action, and risk reduction across the end-to-end data lifecycle.

IBM Big Data strategy

IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy. The company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .

- Smarter Planet was a corporate initiative of IBM, which sought to highlight how government and business leaders were capturing the potential of

smarter systems to achieve economic and sustainable growth and societal progress.

- IBM's data strategy framework consists of six steps: understanding your business objectives, assessing your current state, mapping out data strategy framework, defining your data's target, designing your data architecture, and implementing your data strategy.
- IBM also offers Big Data Analytics solutions, driving advanced analytics with an enterprise-grade, secure, governed, open source-based data lake.
- IBM recommends giving data assets and accelerators top priority, developing a process and culture around data that enables true standardization, re-use, portability, speed to action and risk reduction across the end-to-end data lifecycle.

Introduction to Infosphere

- Infosphere is a term that combines “information” and “sphere” and is analogous to a biosphere.
- It is a metaphysical realm of information, data, knowledge, and communication.
- This realm is populated by informational entities called inforgs (or, informational organisms).
- IBM InfoSphere Information Server is a software platform that provides a single platform for data integration and governance.
- The components in the suite combine to create a unified foundation for enterprise information architectures, capable of scaling to meet any information volume requirements.
- IBM InfoSphere Streams is another software platform that enables the development and execution of applications that process information in data streams.
- InfoSphere Streams enables continuous and fast analysis of massive volumes of moving data to help improve the speed of business insight and decision making.

Introduction to BigInsights

BigInsights is an IBM distribution of Apache Hadoop, a software framework for distributed processing of large data sets across clusters of computers. BigInsights includes several IBM value-added components, such as Big SQL, BigSheets, and Text Analytics, that enhance the capabilities of Apache Hadoop.

Some key features of BigInsights include: - Scalability: BigInsights can handle large volumes of structured and unstructured data. - Flexibility: BigInsights can process data in various formats, including text, images, and videos. - Cost-effectiveness: BigInsights can be deployed on commodity hardware, reducing

the cost of ownership. - Ease of use: BigInsights includes several tools and interfaces that make it easy for users to interact with their data.

BigInsights is used in a variety of industries, including finance, healthcare, and retail, to gain insights from large data sets and make data-driven decisions. It is a powerful tool for businesses looking to harness the power of big data.

Introduction to Big Sheets

- Big Sheets is a user interface program developed for non-technical business users to enable data gathering and analysis .
- It provides new insights by comparing various data from different resources .
- It allows the users to explore the risk factor .
- Big Sheets generates actionable workflow by gathering the information, extracting or analyzing the information, and exploring or visualizing the data in a creative way which enables the users to work .
- Big Sheets can translate untapped data into actionable business insights, which is one of the common requirements .
- Big Sheets is part of IBM's InfoSphere BigInsights platform, which is based on open-source technologies (e.g., Apache Hadoop) and IBM-specific technologies .

Introduction to Big SQL

Big SQL is a high-performance SQL engine that is used to query and analyze large volumes of structured and semi-structured data. It is a part of the IBM BigInsights platform and is built on top of the Apache Hadoop ecosystem. Some of the key features of Big SQL include:

1. **High-performance querying:** Big SQL is designed to provide fast query performance on large volumes of data. It uses advanced query optimization techniques and can take advantage of the parallel processing capabilities of the Hadoop ecosystem.
2. **Support for multiple data sources:** Big SQL can query data stored in a variety of formats and sources, including Hadoop Distributed File System (HDFS), HBase, and other relational databases.
3. **SQL compatibility:** Big SQL supports a wide range of SQL syntax and functions, making it easy for users familiar with SQL to start using it.
4. **Integration with other tools:** Big SQL can be integrated with other tools in the Hadoop ecosystem, such as Hive, Pig, and Spark, to provide a comprehensive data analysis platform.

Overall, Big SQL provides a powerful and flexible tool for querying and analyzing large volumes of data, making it a valuable addition to any big data analytics platform.